

Determinação de ocupação de transportes públicos com base em evidências de comunicação sem fio e aprendizagem de máquina

João Viana, nº 49972 (a49972@alunos.isel.pt)
Tomás Silva, nº 50458 (a50458@alunos.isel.pt)
Gabriel Lemos, nº 50997 (a50997@alunos.isel.pt)

Orientadores: Diego Passos (diego.passos@isel.pt)
Fernanda Passos (fernanda.passos@isel.pt)

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

10 de Julho de 2025

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Determinação de ocupação de transportes públicos com base em evidências de comunicação sem fio e aprendizagem de máquina

49972 João Sousa Viana

50458 Tomás Neves da Silva

50997 Gabriel Lazzarini Lemos

Orientadores: Diego Gimenez Passos

Fernanda Gonçalves de Oliveira Passos

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

10 de Julho de 2025

Resumo

Nos últimos anos, a procura e utilização de transportes públicos tem crescido substancialmente, conduzindo a redes crescentemente sobrelotadas nos períodos de maior procura. Esta exposição contínua a ambientes saturados afeta negativamente a perceção e o bem-estar dos passageiros. Como alternativa aos métodos tradicionais de monitorização de ocupação, que frequentemente sofrem com custos elevados de infraestrutura e problemas de privacidade, este projeto explora uma abordagem de monitorização passiva através de evidências de rede sem fio.

O objetivo principal consistiu na determinação da ocupação em transportes públicos (especificamente no Metropolitano de Lisboa e no sistema de comboios urbanos) utilizando técnicas de aprendizagem de máquina aplicadas a dados de sinais sem fio (Wi-Fi, Bluetooth e redes móveis) captados por dispositivos móveis. Através de uma aplicação *Android*, foram recolhidas milhares de leituras de rede acompanhadas da perceção de lotação dos utilizadores (numa escala de 1 a 5). A avaliação de diferentes algoritmos demonstrou que o modelo *Random Forest* obteve 92,87% de acurácia na classificação.

Foi realizada uma aplicação *full-stack* baseada em *Crowdsourcing* que utiliza o modelo treinado com a informação anonimizada dos utilizadores para inferir a lotação dos transportes da cidade de Lisboa. Esta informação sobre a ocupação é partilhada em tempo real. A abordagem proposta demonstra o potencial da análise passiva de redes sem fio para determinação de ocupação, reduzindo drasticamente os custos e as barreiras de implementação face às soluções tradicionais.

Palavras-chave: Ocupação em tempo real; Aprendizagem de máquina; Telemetria móvel; *Bluetooth*; *Wi-Fi*; Sinais móveis; Metro de Lisboa; *CP*; API REST.

Abstract

In recent years, the demand for and use of public transport have grown substantially, leading to increasingly overcrowded networks during peak periods. This continuous exposure to saturated environments negatively affects passenger perception and well-being. As an alternative to traditional occupancy monitoring methods, which often suffer from high infrastructure costs and privacy issues, this project explores a passive monitoring approach using wireless network evidence.

The main objective was to determine occupancy in public transport (specifically in the Lisbon Metro and the urban train system) using machine learning techniques applied to wireless signal data (Wi-Fi, Bluetooth, and mobile networks) captured by mobile devices. Through an *Android* application, data readings were collected alongside the users' perception of crowdedness (on a scale of 1 to 5). The evaluation of different algorithms demonstrated that the *Random Forest* model achieved a classification accuracy of 92.87%.

A full-stack *Crowdsourcing*-based application was developed to use the trained model with anonymized user information to infer the occupancy of Lisbon's transport network, sharing this information in real time. The proposed approach demonstrates the potential of passive wireless network analysis for occupancy determination, drastically reducing costs and implementation barriers compared to traditional solutions.

Keywords: Real-time occupancy; Machine learning; Mobile telemetry; Bluetooth; Wi-Fi; Cellular signals; Lisbon Metro; REST API.

Uso de Inteligência Artificial

Um *chatbot* específico baseado num *Large Language Model* (LLM), Google Gemini [1], foi ocasionalmente consultado durante o desenvolvimento para auxiliar na correção de erros, na geração de código repetitivo (tais como testes e código análogo), na exploração de soluções alternativas para problemas específicos e na obtenção de ajuda sobre a arquitetura do nosso sistema e a direção do projeto. As suas sugestões foram tratadas como auxílio para o processo de concepção, e não como respostas definitivas, tendo as decisões de implementação sido tomadas com base na nossa compreensão do problema e nas nossas soluções propostas ao projeto.

Índice

1	Introdução	1
1.1	Motivação e Problema	1
1.2	Objetivo do Projeto	1
1.3	Metodologia	2
1.4	Organização do Texto	2
2	Enquadramento	3
2.1	Trabalho relacionado	3
2.2	Sistemas semelhantes	4
2.2.1	APIs	4
3	Coleta de dados	5
3.1	Coleta de dados	5
3.1.1	Base de dados	5
3.1.2	Aplicação de testes da base de dados	5
3.1.3	Dados a serem coletados	5
3.2	Aplicação Android	7
3.3	Estatísticas de Uso	8
3.3.1	Análise de Utilizadores	8
3.3.2	Análise Temporal e Contextual dos Dados	9
4	Modelo e validação dos dados	13
4.1	Pós-processamento de dados	13
4.1.1	Visualização e conversão de dados, derivação de atributos	13
4.1.2	Limpeza de dados comprometidos	16
4.2	Teste do modelo	17
4.2.1	Comparação de algoritmos	17
4.2.1.1	Algoritmo DecisionTable	18
4.2.1.2	Algoritmo J48	18
4.2.1.3	Algoritmo NaiveBayes	19

4.2.1.4	Algoritmo RandomForest	19
4.2.2	Variação de atributos	20
5	Aplicação Final	25
5.1	Aplicação final - Descrição	25
5.2	Prova de conceito - Detecção se o utilizador está dentro do transporte	26
5.2.1	Modelação da Máquina de Estados	26
5.2.2	Mapeamento das Estações	26
5.2.3	Implementação da Aplicação	27
5.2.4	Arquitetura e Módulos da Aplicação	28
5.2.5	Fluxo de Detecção	28
5.3	Casos de uso	29
5.4	Servidor Web em <i>Spring Boot</i>	30
5.4.1	Descrição dos <i>endpoints</i>	31
5.4.1.1	Submissão de telemetria	31
5.4.1.2	Consulta do mapa de ocupação	33
5.4.1.3	Consulta de ocupação de uma estação	33
5.4.2	Modelo de Dados	34
5.4.3	Infraestrutura e <i>implantação</i>	35
5.5	Aplicação Android	36
5.5.1	Funcionamento do <i>frontend</i>	36
5.5.2	Diagrama de serviços UML do sistema completo	39
5.5.3	Algoritmo de deteção de trânsito	40
5.6	Validação de <i>software</i> e testes	40
5.6.1	Testes do servidor	40
5.6.2	Testes da aplicação móvel	41
5.6.3	Cobertura	41
6	Conclusão	42
6.1	Requisitos Cumpridos	42
6.2	Trabalhos Futuros	42

Lista de Figuras

3.1	Ecrã principal da aplicação de recolha de dados.	8
3.2	Ecrã de definições da aplicação de recolha de dados.	8
3.3	Confirmação para garantir integridade dos dados coletados	8
3.4	Mapa de calor de todas as leituras sobre o mapa de Lisboa.	9
3.5	Distribuição do Número de Leituras por Utilizador Único.	10
3.6	Distribuição de Pareto: Volume de Leituras por Utilizador.	10
3.7	Distribuição do Número de Leituras por Tipo de Transporte.	11
3.8	Distribuição do Número de Leituras por Dia da Semana por Transporte. . . .	11
3.9	Distribuição do Número de Leituras por Hora do Dia por Transporte.	12
3.10	Matriz com a Opinião Subjetiva Inicial e Final de cada viagem.	12
4.1	Aplicação de visualização de dados	13
4.2	Informação das leituras de uma viagem	14
4.3	Janela para selecionar onde será salvo o ficheiro	15
5.1	Interface gráfica da aplicação <code>OTP_IsInTransport_PoC</code> a inferir o estado atual do algoritmo de deteção e transições da máquina de estados.	29
5.2	Diagrama entidade-relacionamento desenvolvido em mermaid [2, 3]	35
5.3	Mapa, sem previsões, apresentado no ecrã principal da aplicação final.	37
5.4	Mapa, com previsões, apresentado no ecrã principal da aplicação final.	37
5.5	Ecrã para planear viagem da aplicação final.	38
5.6	Ecrã para planear viagem, com previsão, da aplicação final.	38
5.7	Diagrama de arquitetura do sistema de estimação de ocupação de transporte. .	39
5.8	Diagrama de transição de estados otimizado para o algoritmo de deteção de trânsito.	40

Lista de Tabelas

4.1	Relevância e classificação dos atributos (Mérito Médio e Classificação Média).	18
4.2	Matriz de confusão e métricas de erro. Algoritmo DecisionTable.	19
4.3	Matriz de confusão e métricas de erro. Algoritmo J48.	19
4.4	Matriz de confusão e métricas de erro. Algoritmo NaiveBayes.	20
4.5	Matriz de confusão e métricas de erro. Algoritmo RandomForest.	20
4.6	Matriz de confusão e métricas de erro. Algoritmo RandomForest apenas com horário, localização e estado da rede.	21
4.7	Matriz de confusão e métricas de erro. Algoritmo RandomForest para viagem futura.	21
4.8	Matriz de confusão e métricas de erro. Algoritmo RandomForest sem atributos temporais.	22
4.9	Matriz de confusão e métricas de erro. Algoritmo DecisionTable (dados de treino até 27/06).	22
4.10	Matriz de confusão e métricas de erro. Modelo baseado em fatores temporais e localização (antes de 25 de maio). Algoritmo RandomForest.	23
4.11	Matriz de confusão e métricas de erro. Modelo com fatores temporais e tele- metria de rede (após 25 de maio). Algoritmo RandomForest.	24
4.12	Matriz de confusão e métricas de erro. Modelo enriquecido com atributos adicionais de viagem. Algoritmo RandomForest.	24

Capítulo 1

Introdução

1.1 Motivação e Problema

Nos últimos anos, a procura e utilização de transportes públicos tem crescido substancialmente [4]. Contudo, embora se verifique um reforço na oferta com um maior número de lugares disponíveis, as redes de transporte urbano continuam crescentemente sobrelotadas nos períodos de maior afluência [5, 6].

Esta exposição contínua a ambientes saturados não se traduz apenas num problema logístico, mas traz também um impacto severo no bem-estar dos passageiros. Estudos recentes na área da geografia dos transportes demonstram que a sobrelotação gera elevados níveis de desconforto, stresse e insatisfação, afetando negativamente a perceção subjetiva do tempo de viagem por parte dos utilizadores [7].

Face a este cenário, torna-se imperativo dotar os cidadãos de ferramentas analíticas que permitam mitigar estes efeitos. O âmbito deste projeto foca-se, precisamente, em disponibilizar aos utilizadores uma plataforma interativa de consulta de dados de ocupação em tempo real. Desta forma, fomenta-se uma tomada de decisão informada, permitindo aos passageiros planejar as suas deslocações com base na densidade expectável de cada carruagem ou linha e, consequentemente, otimizar a sua experiência de mobilidade diária.

1.2 Objetivo do Projeto

Por meio de uma aplicação de coleta passiva de evidências sem fio (ondas de rádio), e junto com a utilização de técnicas de aprendizagem de máquina, pretende-se treinar um modelo que seja capaz de determinar a ocupação de transportes públicos (especificamente o Metropolitano de Lisboa e o sistema de comboios urbanos) com base nos sinais sem fio transmitidos por dispositivos móveis pessoais e a qualidade do sinal captado de redes móveis de telefonia (em relação a artefactos introduzidos por um alto número de pessoas).

Também, pretende-se como objetivo secundário o desenvolvimento de uma aplicação que utilize esse modelo alimentado por dados coletados por utilizadores para inferir a lotação dos transportes da cidade de Lisboa, informando os restantes utilizadores em tempo real.

1.3 Metodologia

Este projeto pode ser dividido em 3 partes:

- **Primeira parte:** Esta fase tem como objetivo coletar dados, neste caso através de uma aplicação móvel que vai ser utilizada por voluntários de modo a ter um conjunto de dados representativo para a segunda fase deste projeto.
- **Segunda parte:** Esta etapa utiliza o volume de dados recolhidos com a aplicação desenvolvida na primeira fase de modo a treinar um modelo de aprendizagem de máquina. Este modelo irá receber as métricas de comunicação sem fios recolhidas por utilizadores com o objetivo de prever o nível de ocupação do transporte numa escala de 1 a 5.
- **Terceira parte:** A última fase, de carácter opcional devido ao curto período para o desenvolvimento do projeto, foi o desenvolvimento de uma aplicação móvel baseada em *crowdsourcing* que é dependente da utilização de vários utilizadores em transportes a recolher dados. Estes dados são recolhidos de forma automática e fornecidos ao modelo desenvolvido na fase anterior, permitindo partilhar o estado atual de ocupação dos transportes a outros utilizadores. Outro modelo foi também utilizado para realizar a previsão apenas com base numa data passada pelo utilizador ser possível prever a ocupação futura de um dado transporte.

1.4 Organização do Texto

Este relatório está dividido em 6 capítulos que descrevem a proposta, o enquadramento teórico, e o que foi realizado no âmbito do projeto.

- **Capítulo 1:** Tem como objetivo a introdução e motivação do trabalho.
- **Capítulo 2:** O enquadramento teórico.
- **Capítulo 3:** Tem como responsabilidade descrever a fase de coleta de dados do projeto.
- **Capítulo 4:** Aborda a fase de treino do modelo de aprendizagem de máquina
- **Capítulo 5:** Descreve a aplicação realizada que implementa o modelo desenvolvido.
- **Capítulo 6:** Serve como conclusão e descrição das possíveis melhorias ao trabalho.

Capítulo 2

Enquadramento

Este capítulo está organizado em duas secções onde se descreve o trabalho relacionado e alguns sistemas semelhantes ao sistema desenvolvido.

2.1 Trabalho relacionado

A monitorização eficiente da ocupação em sistemas de transporte público é fundamental para a gestão urbana e para a melhoria da experiência do passageiro [8].

Alguns métodos tradicionais incluem contagem de pessoas por meio de uma abordagem focada em infraestrutura dedicada, nomeadamente: níveis de CO₂ e processamento de imagem [9]. Ambas abordagens sofrem com problemas [9] que incluem custos de implementação e manutenção em larga escala, privacidade (no caso do processamento de imagem) e relativa ineficácia (no caso dos sensores de CO₂).

Surge, assim, a necessidade de explorar soluções tecnológicas alternativas, nomeadamente abordagens provenientes dos meios de comunicação baseados em ondas de rádio aliada à aprendizagem de máquina [10], que permitam obter estimativas precisas em tempo real sobre a densidade populacional nestes espaços.

As pesquisas de vanguarda realizadas sobre estimação de ocupação de forma passiva exploram a análise sobre Informação de Estado do Canal (do inglês, *Channel State Information* ou *CSI*) *Wi-Fi* [11]. A partir disso, é possível verificar a diferença entre um sinal enviado e captado para avaliar o nível de interferência que é causado por uma multidão de pessoas na Informação de Estado do Canal [10, 12].

Outra diferença entre este trabalho e os de ponta é o aprendizagem de máquina. Enquanto foi desenvolvido um modelo criado a partir de algoritmos de aprendizagem de máquina clássicos (e.g., *Random Forest* [13], *SVM* [14]), na vanguarda, o utilizado são Redes Neurais. A principal vantagem é observada na seleção de atributos para o treino do modelo. Nas pesquisas é usada a Aprendizagem Profunda (do inglês, *Deep Learning*) para extrair características

diretamente das flutuações brutas e não-lineares da Informação de Estado do Canal de forma automática [12, 15].

2.2 Sistemas semelhantes

Alguns sistemas de estimação de ocupação implementados em sistemas de transportes públicos em Portugal e em outras cidades incluem:

- *Comboios de Portugal* - Sistema baseado em dados de operação e sensores [16];
- *Google Maps* - Sistema assente em *crowdsourcing* passivo, e dados de geolocalização por GPS recolhidos de utilizadores em tempo real [17];
- *TfL Go* - Sistema baseado no processamento de dados de validação de cartões de transporte nas barreiras físicas de acesso [18].

Ao contrário destas soluções, que dependem ou de sensores nas carruagens (CP), ou de um número massivo de utilizadores com GPS ativo (Google), ou de infraestruturas físicas de controlo de acessos (TfL), a abordagem desenvolvida neste projeto foca-se na monitorização passiva local através de evidências de rede sem fio, reduzindo drasticamente os custos e as barreiras de implementação.

2.2.1 APIs

Embora a CP (Comboios de Portugal) não disponibilize uma API (*Application Programming Interface*) pública oficial, existem diversas aplicações baseadas em engenharia reversa. Este método permite o consumo de *endpoints* privados utilizados pelas plataformas digitais da operadora, tornando possível obter dados como a listagem de estações, detalhes sobre a frota de comboios e informações de viagens, como a localização e os horários previstos.

Adicionalmente, a CP disponibiliza dados no formato padrão GTFS (*General Transit Feed Specification*) [19]. Apesar de este formato exigir uma lógica de implementação e processamento mais complexos, por lidar maioritariamente com dados estáticos e planeamento de longo prazo, representa o canal oficial de acesso à informação da infraestrutura e operação da empresa.

O Metropolitano de Lisboa, ao contrário da CP, disponibiliza uma API pública oficial [20], que permite o acesso direto a dados estruturados sobre as estações e o estado de funcionamento das linhas, e informações sobre o tempo de espera em cada estação.

Existem várias aplicações que implementam as APIs acima descritas, como por exemplo, a *CP Realtime* [21], e o *Próximo Metro* [22].

Capítulo 3

Coleta de dados

3.1 Coleta de dados

Foi criada uma aplicação Android para realizar a coleta de dados sem-fio possíveis de serem captados por um *smartphone* moderno (definido na proposta do projeto como meio da coleta), necessários para o treino do modelo de aprendizagem de máquina.

3.1.1 Base de dados

A base de dados escolhida foi a *Google Firebase Firestore* [23] pertencente à plataforma de desenvolvimento *Firebase*. Esta foi escolhida por conta do suporte provido a APIs nativas do ambiente de desenvolvimento *Android*, *Android Studio* [24], da própria *Google*.

Também, era desnecessária a utilização de uma base de dados relacional já que o requisito principal era apenas o armazenamento veloz de dados em média escala (algumas milhares de escritas).

3.1.2 Aplicação de testes da base de dados

Com o intuito de testar o funcionamento e fluxo de autenticação da API para realizar operações de escrita, armazenar os dados coletados, foi desenvolvida uma aplicação simples chamada `mock_db_test.kt`.

São criados um documento e uma coleção, é feito um pedido de escrita, e é escrito o resultado na saída padrão da aplicação.

Apesar de não ser um requisito da aplicação realizada na fase de coleta de dados, também foi testado o fluxo de chamadas à API para obter os dados armazenados.

3.1.3 Dados a serem coletados

Após realizarmos uma pesquisa sobre trabalhos relacionados [9, 10], limitações do *Android* e consultarmos os professores orientadores, chegamos a lista de dados possíveis e úteis a serem

coletados para realizar o treinamento de um modelo de previsão de ocupação.

Todas as leituras estão associadas a uma viagem específica realizada por algum utilizador voluntário. A viagem em si, somente possui a informação de data e hora, tipo de transporte utilizado (metro ou comboio), e um valor booleano que representa se o utilizador confirmou que a viagem está válida (se este permaneceu dentro do transporte especificado durante toda a leitura).

Para cada viagem, está associada uma coleção de leituras, coletadas em intervalos de 32 segundos devido às restrições de sondagem de rede *Wi-Fi* próximas do *Android* [25], com os seguintes dados fornecidos pelas APIs de acesso ao *hardware* de rede do dispositivo [26].

- **Contagem de dispositivos Bluetooth:** O número total de dispositivos únicos detetados pelo recetor *Bluetooth*.
- **5 dispositivos com a maior intensidade de sinal Bluetooth:** Coletamos o indicador de intensidade dos sinais mais fortes.
- **Contagem de redes Wi-Fi:** O número total de redes de *Wi-Fi* detetadas.
- **5 redes Wi-Fi com maior intensidade:** Coletamos os maiores indicadores de intensidade das redes Wi-Fi detetadas.
- **Reference Signal Received Power (RSRP):** [27] Indicador da potência do sinal recebido pela antena de dados móveis do sistema. Medido em decibel-miliwatt (dBm).
- **Reference Signal Signal-to-Noise Ratio (RSSNR):** [27] Mede a relação entre o sinal e o ruído juntamente com a interferência de outras antenas de telefonia. Medido em decibéis (dB).
- **Reference Signal Received Quality (RSRQ):** [27] Combina a força do sinal com o nível total de interferência na banda de frequência, representando a qualidade geral do sinal. Medido em dB.

Foram coletados os dados de conexão por meio de 5 pedidos *ping* para o servidor DNS (*Domain Name Server*) público da *Google* com o endereço 8.8.8.8 [28]. Este servidor foi escolhido pela sua ampla disponibilidade e confiabilidade.

- **Média da latência:** A média da latência entre o pedido e a resposta dos 5 pedidos *ping*
- **Desvio padrão da latência:** Uma medida de dispersão estatística que indica o quão próximo os dados estão da média. Possível correlação com um ambiente muito cheio.
- **Percentagem de perda de pacotes:** Um valor que acompanha a resposta do comando *Linux ping* [29]. Indica quantos pacotes não foram recebidos em resposta aos pedidos

feitos.

Outros dados gerais coletados sobre cada leitura:

- **Latitude e Longitude (Localização):** Como o modelo será treinado especificamente para a área metropolitana da cidade de Lisboa, é um dado possivelmente relevante.
- **Timestamp (Horário):** Definido com o número de segundos desde 1 de janeiro de 1970, é útil pelo facto de haver horários com lotação muito superior a outros.
- **Lotação subjetiva de acordo com o utilizador (1 - 5):** Utilizado para que, no momento do treino do modelo, os dados sejam correlacionados com a sensação de lotação de um passageiro. Importante para o teste do modelo. Cada leitura recebe a última avaliação subjetiva do voluntário.

3.2 Aplicação Android

A aplicação *Android* desenvolvida em *Kotlin* nesta fase do projeto teve como objetivo principal realizar a recolha dos dados mencionados na secção anterior. Procurou-se criar uma interface intuitiva e de fácil utilização, de modo a aumentar ao máximo o número de voluntários e incentivar a sua utilização recorrente.

A interface da aplicação é composta por dois ecrãs. No principal, como mostrado na Figura 3.1, é possível iniciar uma viagem, selecionando previamente o meio de transporte a ser utilizado (metro ou comboio) e classificando de 1 a 5 a ocupação atual do transporte, de acordo com a perceção do utilizador. Ao iniciar uma viagem são feitas leituras periódicas de dados a cada 30 segundos. Durante a viagem o voluntário pode atualizar a sua avaliação de lotação sem que haja a necessidade de interromper e reiniciar a viagem atual. Por fim os dados recolhidos são então armazenados na base de dados não relacional *Google Firebase Firestore* [23].

O segundo ecrã permite ao utilizador configurar as definições da aplicação. Neste menu, mostrado na Figura 3.2 é possível escolher o idioma (português ou inglês), ajustar a duração padrão de uma viagem (inicialmente definida em 15 minutos), e o intervalo de tempo entre notificações.

Um dos problemas no desenvolvimento desta aplicação foi a dependência da ação dos voluntários para iniciar e terminar o registo apenas quando efetivamente se encontravam dentro do transporte. Para minimizar possíveis enganos, ou esquecimentos, enviamos notificações periódicas para os utilizadores para lembrar que a aplicação se encontra activa e para sugerir a alteração da avaliação subjetiva da lotação, caso necessário. Mesmo com este aviso existe o risco de o utilizador sair do transporte e não cancelar de imediato a recolha dos dados. Por este motivo, ao cancelar a viagem há a opção de invalidar a viagem caso o utilizador não tenha permanecido no veículo durante a totalidade do tempo, conforme mostrado na Figura 3.3.

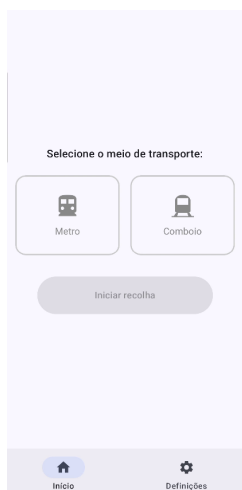


Figura 3.1: Ecrã principal da aplicação de recolha de dados.



Figura 3.2: Ecrã de definições da aplicação de recolha de dados.

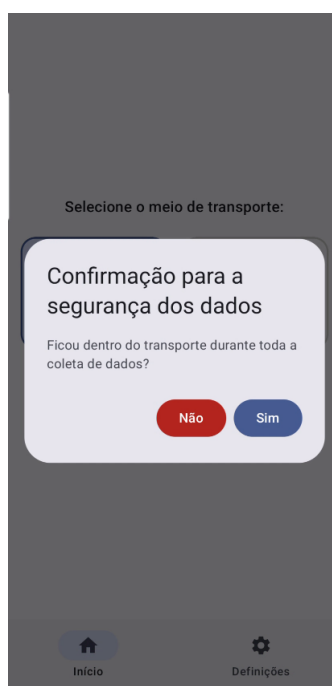


Figura 3.3: Confirmação para garantir integridade dos dados coletados

3.3 Estatísticas de Uso

3.3.1 Análise de Utilizadores

A aplicação foi distribuída por vários colegas, amigos, e professores, além de ter sido feito um anúncio a todos os alunos de LEIC (Licenciatura de Engenharia Informática e Computadores) do ISEL. Até a data, temos um total de 17 utilizadores, que juntamente realizaram um total de 7686 leituras. A Figura 3.4 mostra a localização de todas as leituras através de um mapa de calor. Na Figura 3.5 é possível observar o número de contribuições por utilizador. No total

os utilizadores realizaram 551 viagens, cerca de 32 viagens por utilizador distinto. Em termos de uso, em média cada utilizador contribuiu com 4 horas e 1 minuto de leituras.

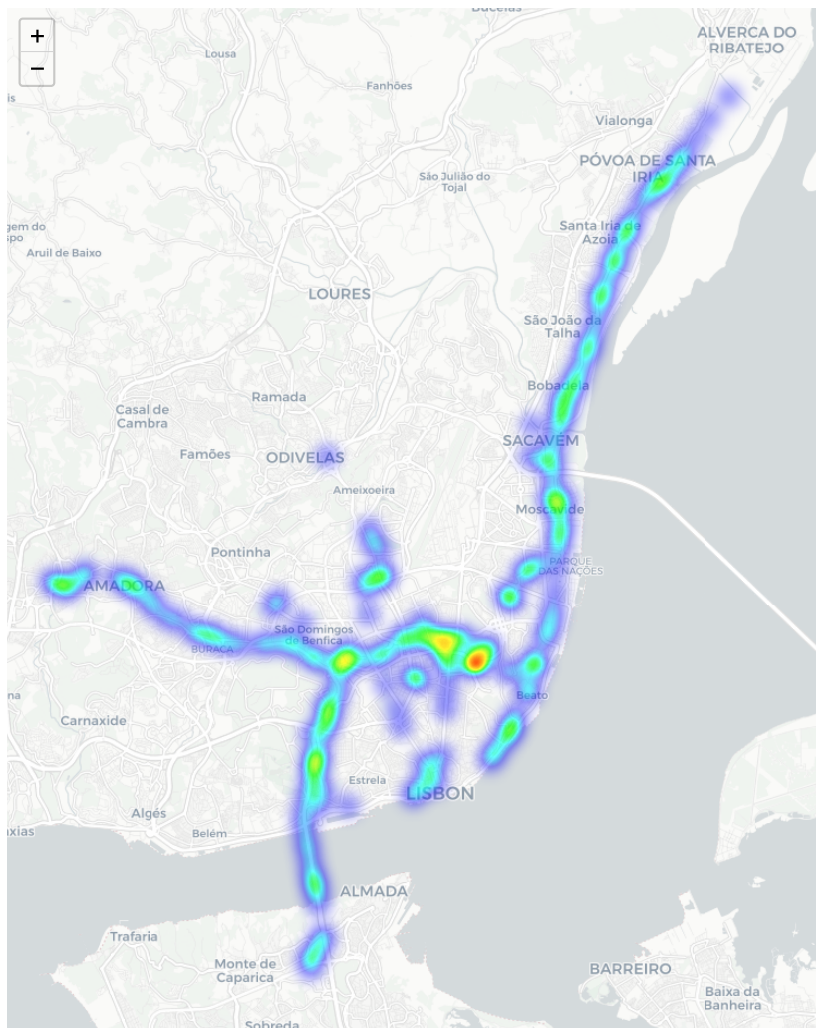


Figura 3.4: Mapa de calor de todas as leituras sobre o mapa de Lisboa.

De notar que os maiores contribuidores, em leituras, para a recolha dos dados, se concentram em poucos utilizadores, onde os 2 maiores acumulam 68.4% de todas as leituras, e onde cerca de 75% dos dados se acumulam em 25% dos utilizadores, como se pode observar na Figura 3.6, um comportamento estatístico que se alinha com o Princípio de Pareto [30]. Isto mostra que para o funcionamento da aplicação final de *crowdsourcing* cerca de 20% dos utilizadores seriam suficientes para fornecer 80% dos dados necessários, no entanto também mostra a significância que uma opinião subjetiva de um certo utilizador, pode ter a enviesar os dados.

3.3.2 Análise Temporal e Contextual dos Dados

A Figura 3.7 mostra a distribuição das leituras por tipo de transporte, em que podemos ver que o comboio tem cerca de 38% mais leituras do que o metro.

As Figuras 3.8 e 3.9 descrevem o número de leituras por dia da semana, e por hora, por tipo

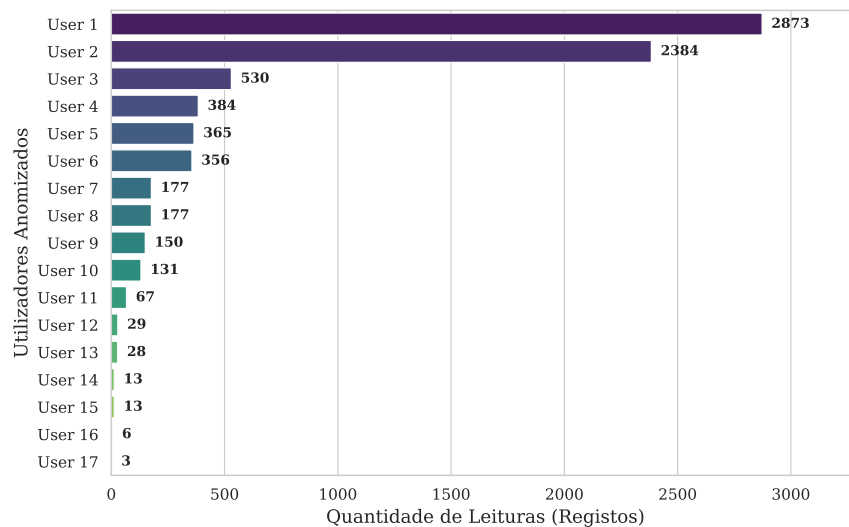


Figura 3.5: Distribuição do Número de Leituras por Utilizador Único.

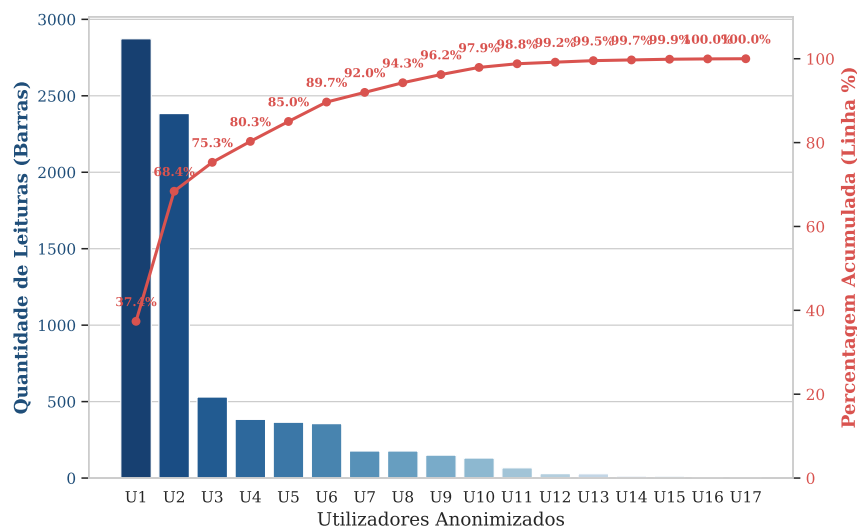


Figura 3.6: Distribuição de Pareto: Volume de Leituras por Utilizador.

de transporte.

Com base nestes gráficos, conseguimos identificar em que períodos, por tipo de transporte, pode existir uma falta de dados, levando a uma menor precisão do modelo, na tentativa de previsão.

A Figura 3.10 mostra as transições de opinião subjetiva no início e final de cada viagem, mostrando que embora a maioria se situe na diagonal, que mostra que a viagem terminou com a mesma ocupação subjetiva com que iniciou, existem várias viagens onde foi alterada, fator vital para o treinamento do modelo.

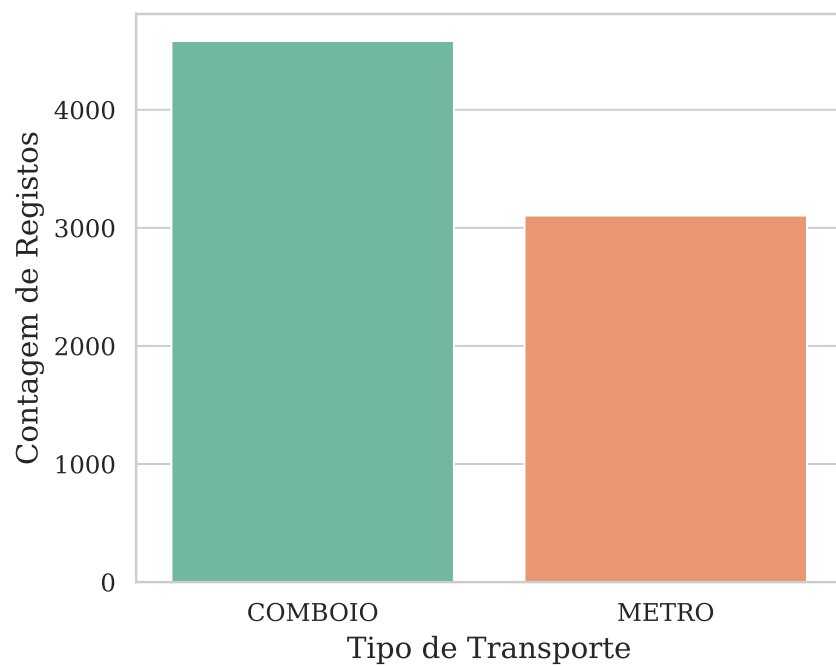


Figura 3.7: Distribuição do Número de Leituras por Tipo de Transporte.

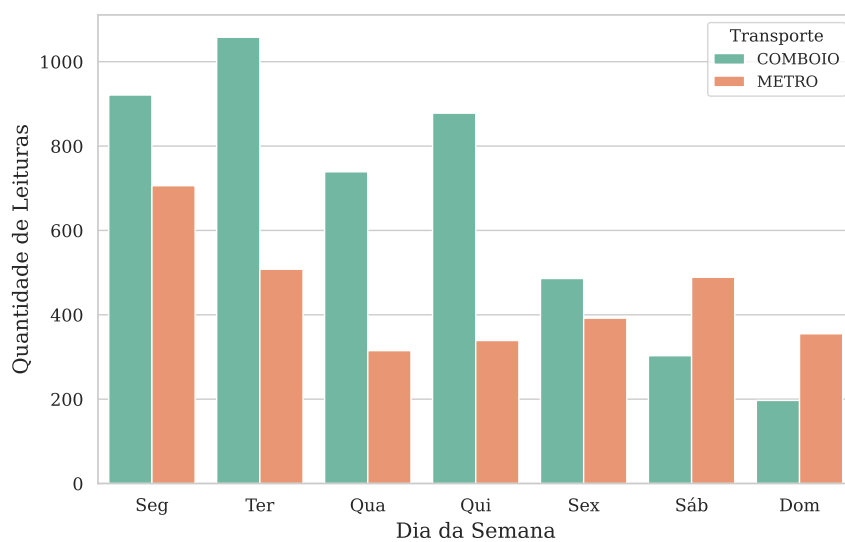


Figura 3.8: Distribuição do Número de Leituras por Dia da Semana por Transporte.

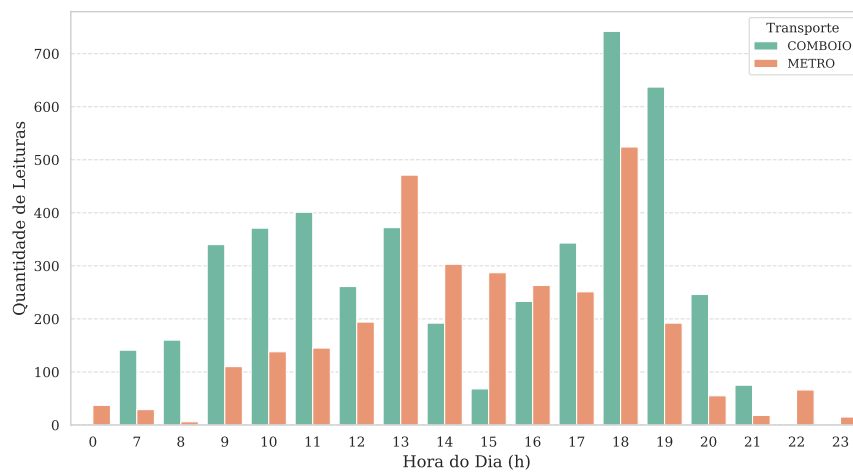


Figura 3.9: Distribuição do Número de Leituras por Hora do Dia por Transporte.

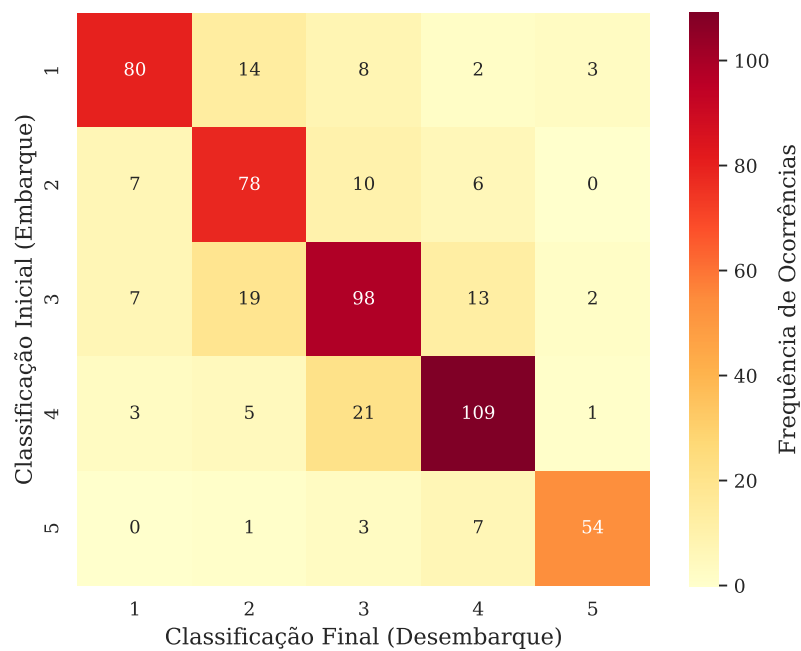


Figura 3.10: Matriz com a Opinião Subjetiva Inicial e Final de cada viagem.

Capítulo 4

Modelo e validação dos dados

4.1 Pós-processamento de dados

4.1.1 Visualização e conversão de dados, derivação de atributos

Durante a fase de coleta de dados desenvolveu-se uma aplicação em *Kotlin* [31] com interface gráfica, realizada por meio de uma biblioteca do kit de desenvolvimento de GUI (*Graphical User Interface*) nativo da linguagem *Java* [32], *Swing* [33]. A aplicação foi nomeada *OTPCD Data Viewer* e cumpre três principais objetivos.

O primeiro destes objetivos foi ter alguma maneira prática e rápida para visualizar os dados armazenados no *Google Firebase Firestore* ordenados pela data em que foram coletados pela nos aplicação de coleta de dados *Android*.

Pode-se ver na Figura 4.1 a aplicação em funcionamento.

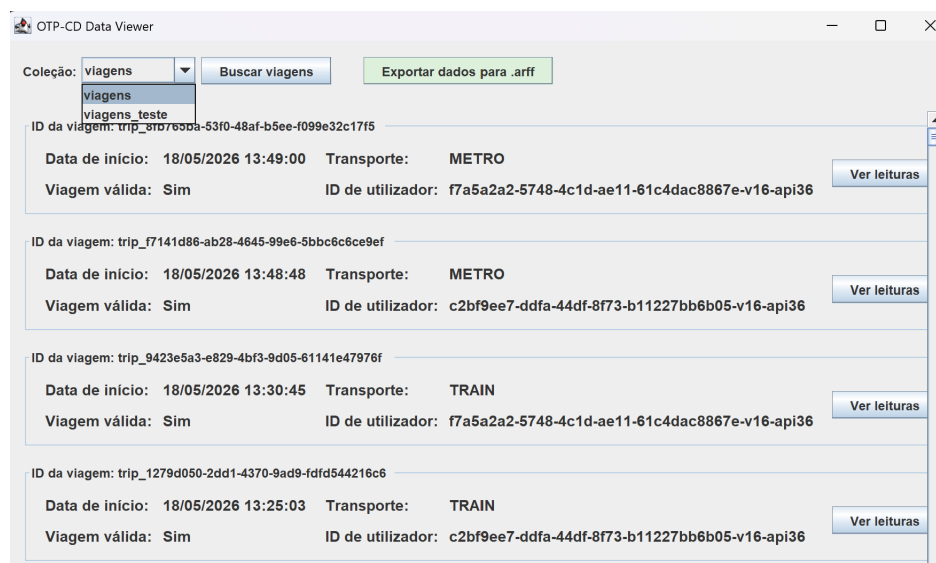


Figura 4.1: Aplicação de visualização de dados

A aplicação permite também escolher entre qual coleção de dados visualizar. A utilizada em

contexto de produção ("viagens") e em contexto de desenvolvimento para teste do funcionamento da aplicação de coleta de dados ("viagens_teste").

No ecrã primário da aplicação, como pode-se ver na Figura 4.1 é listada cada viagem feita de forma individual, incluindo o horário em que foi realizada, o tipo de transporte, se o utilizador verificou-a como válida ou não, o ID anonimizado do utilizador que realizou a viagem e um botão para visualizar todas as leituras feitas durante a viagem.

Ao clicar no botão "Ver leituras", abre-se uma nova janela com a informação das leituras realizadas em sua totalidade, como pode ser visualizado na Figura 4.2.

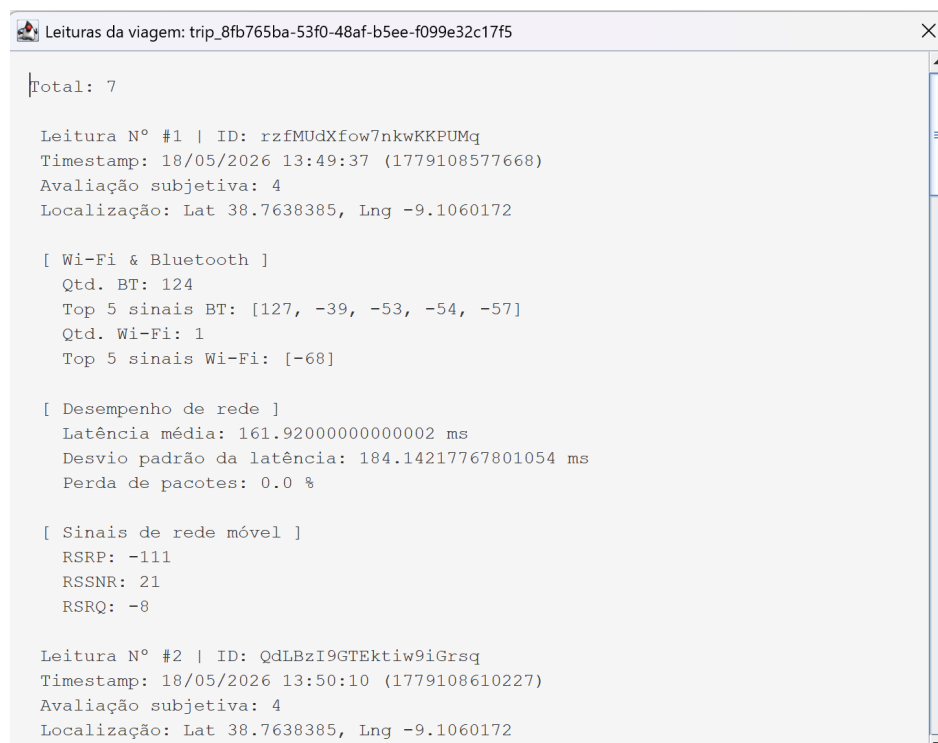


Figura 4.2: Informação das leituras de uma viagem

Além dos dados coletados citados no Capítulo 3, também é possível verificar o número total de leituras realizadas durante a viagem como é visível na Figura 4.2 na secção "Total".

O segundo dos objetivos cumpridos por esta aplicação é a conversão dos dados armazenados na base de dados não relacional *Google Firebase Firestore* em dados tabularizados para a fase de aprendizagem de máquina no formato *.arff* [34] (*Attribute-Relation File Format*) que é o tipo de ficheiro aceito pela aplicação *WEKA* [35].

Como é possível visualizar no centro-topo da janela na Figura 4.1, há um botão "Exportar dados para *.arff*". A ser pressionado, é chamada a função nativa da biblioteca *Swing JFileChooser()* para selecionar dentro do sistema de ficheiros onde serão salvos os dados extraídos em *.arff* com o nome predefinido, *dataset_otpcd_viagens.arff*, como se pode verificar

na Figura 4.3

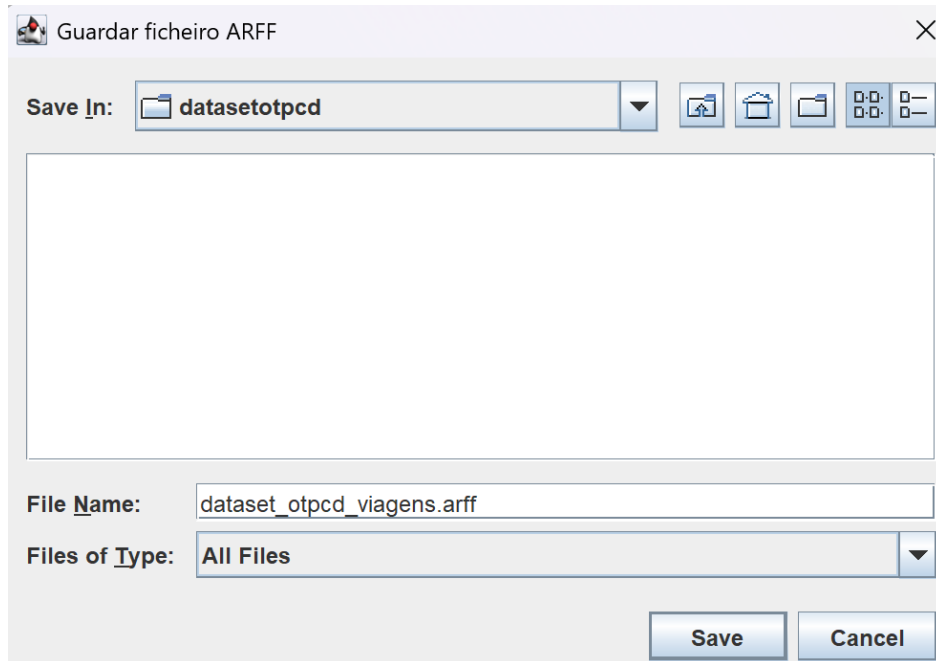


Figura 4.3: Janela para selecionar onde será salvo o ficheiro

Os dados serão convertidos para o formato *.arff* com todos os atributos corretamente selecionados, pronto para ser carregado na aplicação *WEKA*.

O terceiro objetivo realizado por esta aplicação foi a derivação de novos atributos com base em dados já coletados e armazenados. Estes novos dados foram extraídos com a função de auxiliar os algoritmos de aprendizagem automática utilizados no desenvolvimento do modelo a interpretar matematicamente informações sobre a viagem realizada. Especificamente por meio de valores *booleanos* que representam abstrações humanas que normalmente influenciam a ocupação.

Os novos valores foram todos derivados do atributo existente que representa a data e hora, o *timestamp* (um *Unix Timestamp* [36]).

Os atributos derivados foram os seguintes:

- **dayOfWeek:** Retira o dia da semana por meio da biblioteca nativa Java, *ZonedDateTime* e o campo pré-existente *dayOfWeek.value*. Representado em um inteiro entre 1 e 7.
- **hour:** Também é derivado a partir da biblioteca Java, mas por meio do campo *hour*. Representado em um inteiro entre 0 e 23.
- **isWeekend:** Abstração booleana derivada do atributo *dayOfWeek*.
- **isRushHour:** Outra abstração booleana derivada do atributo *hour*, especificamente definida entre 7 e 9 ou entre 17 e 19.

É importante também ressaltar que a aplicação não foi desenvolvida para uso público, e que apesar de seu código-fonte (que não contém nenhuma informação sensível) estar armazenado na plataforma *GitHub* [37], o seu funcionamento depende de uma chave de *API* do sistema do *Google Firebase* com permissões de administrador, em variável de ambiente. Esta chave foi apenas compartilhada entre os autores do projeto.

4.1.2 Limpeza de dados comprometidos

Como citado no Capítulo 3, quando uma viagem é criada, há um indicador (um campo *booleano*) que representa se a viagem foi válida ou não. Ou seja, se o utilizador permaneceu dentro da carruagem do transporte durante toda a viagem.

Para tentar recuperar parte dos dados, criou-se um *Script* na linguagem *Python* [38] nomeado *non-valid-data-cleanup.py* (do inglês, limpeza de dados não-válidos). O *script* executa um algoritmo que tenta identificar a partir de qual momento as leituras coletadas passam a ser inválidas.

O processamento das viagens inválidas é realizado por meio do seguinte algoritmo:

1. Extração de velocidade

Numa primeira fase, o algoritmo ordena os registos cronologicamente por utilizador e calcula as métricas de movimento essenciais entre cada leitura consecutiva. A distância percorrida é calculada através da Fórmula de *Haversine* [39], que considera a curvatura da Terra para obter a distância em metros entre dois pontos geográficos (latitude e longitude). Com base nesta distância e na diferença temporal entre os registos, é inferida a velocidade de deslocação em quilómetros por hora.

2. Divisão de viagens

Para garantir que análises contíguas pertencem à mesma deslocação, o sistema identifica quebras temporais. Se o intervalo entre duas leituras consecutivas do mesmo utilizador for superior a 10 minutos, o algoritmo assume que se trata de uma nova viagem, atribuindo-lhe um identificador único, já que as viagens extraídas em *.arff* não contém identificação de viagem como atributo.

3. Recorte baseado no meio de transportes

A fase final aplica lógicas de recorte distintas conforme o tipo de transporte, assumindo diferentes padrões de movimento quando o utilizador sai da carruagem.

- **Comboio:** O algoritmo procura picos de velocidade superiores a 30km/h. Como cada leitura é feita de 30 em 30 segundos, a viagem é recortada logo após o último pico de alta velocidade registado, assumindo que, a partir desse momento, o comboio parou. O recorte é efetuado com uma margem de segurança de dois registos após a imobilização.
- **Metro:** Dado que a informação de localização dentro da rede de Metro é muito limitada, o algoritmo foca-se em identificar o padrão de caminhada. Especificamente quando são calculadas velocidades entre 1,5 km/h e 8,0 km/h. O momento de recorte é definido consideravelmente antes deste padrão para ter a certeza de não considerar dados coletados dentro da estação.

Os dados recuperados e limpos são, por fim, exportados de volta para o formato *.arff*, garantindo a compatibilidade com as ferramentas de aprendizagem de máquina utilizadas nas fases seguintes do projeto.

4.2 Teste do modelo

Antes da construção dos modelos, realizámos uma análise de relevância dos atributos para avaliar a contribuição de cada variável para o conjunto de dados. Utilizámos o método *InfoGainAttributeEval*, que calcula a importância média de cada atributo para o conjunto de dados. Como se observa na Tabela 4.1, todos os atributos apresentam um valor positivo de relevância, não existindo variáveis com mérito nulo. Consequentemente, optámos por manter a totalidade dos atributos no treino dos modelos, uma vez que, mesmo aqueles com menor impacto individual, podem conter padrões que ajudam os algoritmos na classificação da ocupação.

4.2.1 Comparação de algoritmos

Com base no conjunto de dados apresentados na Tabela 4.1, procedeu-se à construção de vários modelos de aprendizagem de máquina utilizando a ferramenta WEKA [35]. Todos os testes em que não é explícito o método de treino do modelo, foram realizados recorrendo à metodologia de validação cruzada com 10 partições (*10-fold cross-validation*). Neste processo, o conjunto dos dados é dividido em 10 subconjuntos. São feitas 10 iterações e, em cada iteração, uma partição diferente é reservada para testar o modelo, enquanto as restantes 9 são utilizadas para o seu treino. Assim garante-se que todos os dados são utilizados tanto para construir o modelo como para o validar. O objetivo consistiu em prever o nível de ocupação dos transportes, classificado numa escala subjetiva de 1 a 5, que foi escolhida por ter o mesmo

Tabela 4.1: Relevância e classificação dos atributos (Mérito Médio e Classificação Média).

Atributo	Mérito Médio	Classificação Média
hour	0.332 ± 0.003	1.0 ± 0.0
bluetoothCount	0.127 ± 0.003	2.0 ± 0.0
longitude	0.069 ± 0.005	3.3 ± 0.46
wf_signal_1	0.051 ± 0.005	4.7 ± 1.19
latitude	0.055 ± 0.013	5.1 ± 1.51
dayOfWeek	0.048 ± 0.001	5.7 ± 0.9
wf_signal_5	0.045 ± 0.002	6.5 ± 0.92
wifiCount	0.037 ± 0.004	9.2 ± 1.17
bt_signal_5	0.036 ± 0.001	9.7 ± 1.19
wf_signal_2	0.036 ± 0.005	10.0 ± 1.61
wf_signal_3	0.035 ± 0.004	10.3 ± 1.85
bt_signal_4	0.032 ± 0.001	11.7 ± 0.78
wf_signal_4	0.031 ± 0.006	12.1 ± 2.3
bt_signal_3	0.029 ± 0.001	13.7 ± 0.46
bt_signal_2	0.024 ± 0.000	15.0 ± 0.0
bt_signal_1	0.016 ± 0.002	16.0 ± 0.0
latencyAvg	0.014 ± 0.001	17.3 ± 0.46
transportType	0.013 ± 0.001	17.9 ± 0.7
rsrq	0.012 ± 0.001	18.8 ± 0.4
rsrp	0.007 ± 0.000	20.4 ± 0.49
packetLoss	0.006 ± 0.001	21.4 ± 0.49
latencyStdDev	0.005 ± 0.005	21.6 ± 1.36
rssnr	0.004 ± 0.000	22.6 ± 0.49

nível de resolução da escala Likert [40] amplamente utilizada em sondagens de opinião. Salvo indicação em contrário, todos os resultados apresentados nesta secção baseiam-se em modelos treinados com o conjunto de dados recolhido até ao dia 21 de maio de 2026.

4.2.1.1 Algoritmo DecisionTable

A análise dos resultados da Tabela 4.2 evidencia métricas de avaliação aceitáveis, especialmente considerando a natureza subjetiva da variável alvo. Acredita-se que ainda existe margem para melhorar, não só por ser o primeiro algoritmo testado, mas também por este algoritmo não utilizar todos os atributos fornecidos.

4.2.1.2 Algoritmo J48

Observa-se na Tabela 4.3 uma ligeira melhoria no desempenho face ao modelo anterior, embora os resultados continuem a sugerir potencial para melhorar.

Tabela 4.2: Matriz de confusão e métricas de erro. Algoritmo DecisionTable.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	745	21	1	9	1	Número total de leituras	4813
2	67	803	39	31	0	Número de previsões erradas	689
3	48	40	1051	142	1	Percentagem de erro	14,32%
4	22	6	130	1118	58	Percentagem de acerto	85,68%
5	17	1	4	51	407	Erro médio quadrático	0,309163
						Erro absoluto médio	0,186578
						Erro médio com sinal	-0,038229794
						Média do F1-Score	0,857

Tabela 4.3: Matriz de confusão e métricas de erro. Algoritmo J48.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	710	44	17	5	1	Número total de leituras	4813
2	31	842	46	21	0	Número de previsões erradas	558
3	26	54	1115	75	12	Percentagem de erro	11,59%
4	2	19	83	1182	48	Percentagem de acerto	88,41%
5	0	6	14	54	406	Erro médio quadrático	0,208602
						Erro absoluto médio	0,144608
						Erro médio com sinal	-0,00664866
						Média do F1-Score	0,884

4.2.1.3 Algoritmo NaiveBayes

O desempenho deste algoritmo resumido na Tabela 4.4 revelou-se bastante insatisfatório. Tratando-se de uma classificação com cinco opções possíveis (1 a 5), um modelo de previsão puramente aleatória teria uma acurácia de cerca de 20%. Os resultados obtidos por este algoritmo são ligeiramente melhores que os desse modelo aleatório.

4.2.1.4 Algoritmo RandomForest

Este modelo obteve os melhores resultados, como é possível observar na Tabela 4.5 atingindo uma acurácia na ordem dos 92%, um valor muito bom tratando-se de uma previsão de uma avaliação subjetiva. A análise da matriz de confusão mostra que a grande maioria das classificações incorretas falhou por apenas um valor (por exemplo, prever o nível 3 quando o real era o 4). Esta ligeira discrepância é justificável pelo teor subjetivo da avaliação da lotação.

Tabela 4.4: Matriz de confusão e métricas de erro. Algoritmo NaiveBayes.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	181	506	29	16	42	Número total de leituras	4813
2	52	794	22	39	33	Número de previsões erradas	3229
3	57	809	86	219	111	Percentagem de erro	67,09%
4	21	694	72	406	141	Percentagem de acerto	32,91%
5	26	217	19	101	117	Erro médio quadrático	10,267816
						Erro absoluto médio	2,958446
						Erro médio com sinal	-0,378558072
						Média do F1-Score	0.293

Tabela 4.5: Matriz de confusão e métricas de erro. Algoritmo RandomForest.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	737	31	4	3	2	Número total de leituras	4813
2	23	861	49	7	0	Número de previsões erradas	343
3	8	23	1188	59	4	Percentagem de erro	7,13%
4	2	11	49	1257	15	Percentagem de acerto	92,87%
5	0	2	10	41	427	Erro médio quadrático	0,153335
						Erro médio absoluto	0,108041
						Erro médio com sinal	-0,001038853
						Média do F1-Score	0.929

4.2.2 Variação de atributos

Após a avaliação de outros modelos, que apresentaram um desempenho inferior ao Random-Forest, optou-se por focar o estudo neste algoritmo, realizando testes de variação dos atributos de modo a adequar o modelo aos diferentes cenários práticos da aplicação.

O primeiro cenário testado restringiu os dados fornecidos apenas à localização, ao horário e ao estado da rede (latência e perda de pacotes). Este teste tem muita importância prática para um possível trabalho futuro dado que, numa aplicação Web, estes são os únicos dados passíveis de ser obtidos devido às restrições dos navegadores. Uma aplicação Web seria a forma mais simples de implementar o nosso sistema no ecossistema iOS, o que poderá ser um objetivo futuro.

Conforme antecipado, observa-se com os resultados na Tabela 4.6 uma degradação nas métricas de avaliação face ao modelo completo. Contudo, para além de mais simples, esta seria uma forma eficaz de implementar o nosso sistema no ecossistema iOS.

Tabela 4.6: Matriz de confusão e métricas de erro. Algoritmo RandomForest apenas com horário, localização e estado da rede.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	745	24	6	1	1	Número total de leituras	4813
2	40	842	51	7	0	Número de previsões erradas	486
3	13	60	1114	89	6	Percentagem de erro	10,10%
4	1	4	73	1211	45	Percentagem de acerto	89,90%
5	0	1	10	54	415	Erro médio quadrático	0,137752
						Erro absoluto médio	0,112404
						Erro médio com sinal	-0,006856431
						Média do F1-Score	0,899

Testámos também para o caso de uma viagem futura. Neste cenário, a leitura de dados de comunicação sem fios é impossível. O sistema tem de utilizar exclusivamente as variáveis possíveis: a localização e a hora estimada da viagem. Testou-se o algoritmo para este caso.

Tabela 4.7: Matriz de confusão e métricas de erro. Algoritmo RandomForest para viagem futura.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	752	20	4	1	0	Número total de leituras	4813
2	20	848	66	6	0	Número de previsões erradas	617
3	11	66	1061	140	4	Percentagem de erro	12,82%
4	0	4	149	1125	56	Percentagem de acerto	87,18%
5	0	3	7	60	410	Erro médio quadrático	0,157282
						Erro absoluto médio	0,137336
						Erro médio com sinal	-0,007271972
						Média do F1-Score	0,872

Os resultados da Tabela 4.7 demonstraram-se positivos, suficientes para que esta funcionalidade integre a aplicação final. Esta restrição permite um mecanismo de incentivo: utilizadores passivos, que por alguma razão não partilhem os seus dados de sensores com a plataforma, teriam acesso apenas a estas estimativas, enquanto que quem contribui com métricas em tempo real, seria recompensado com uma estimativa em tempo real dos transportes.

Uma preocupação que o teste anterior levantou, visto que apenas com informações temporais e de localização houve bons resultados, prendeu-se com uma possível grande dependência dos outros modelos em relação ao fator temporal (hora do dia e dia da semana). Embora sejam indicadores muito bons na rotina diária de transportes, podem induzir o modelo em erro em situações atípicas, como eventos desportivos, concertos ou greves. Para testar estes casos,

treinou-se um modelo apenas com os dados de comunicação sem fios.

Tabela 4.8: Matriz de confusão e métricas de erro. Algoritmo RandomForest sem atributos temporais.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	544	82	76	71	4	Número total de leituras	4813
2	80	554	149	142	15	Número de previsões erradas	1620
3	47	90	905	225	15	Percentagem de erro	33,66%
4	49	63	180	1014	28	Percentagem de acerto	66,34%
5	27	33	76	168	176	Erro médio quadrático	0,973613
						Erro absoluto médio	0,512778
						Erro médio com sinal	-0,004155412
						Média do F1-Score	0,658

Apesar de as métricas globais não serem tão elevadas, este classificador mostra desempenho suficiente para casos excecionais.

Como referido, os modelos acima foram treinados com dados recolhidos até 21 de maio. Com o objetivo de aferir a robustez dos modelos de aprendizagem face a dados mais recentes, procedemos a uma validação adicional comparando os resultados obtidos com os dados recolhidos até 21 de maio (Tabela 4.2) e os resultados com o conjunto de dados mais recente, recolhidos até 27 de junho (Tabela 4.9). Foi usado o algoritmo *DecisionTable*, por ser o que demonstrou uma diferença mais expressiva com os dados atualizados, porém, todos os algoritmos mostraram um desempenho pior com estes dados.

Tabela 4.9: Matriz de confusão e métricas de erro. Algoritmo DecisionTable (dados de treino até 27/06).

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	1152	99	166	24	2	Número total de leituras	7550
2	243	906	263	42	23	Número de previsões erradas	2379
3	238	157	1381	215	32	Percentagem de erro	31,51%
4	147	67	320	1234	75	Percentagem de acerto	68,49%
5	45	18	79	124	498	Erro médio quadrático	0,097748
						Erro absoluto médio	0,068874
						Erro médio com sinal	-0,13325
						Média do F1-Score	0,685

Os resultados evidenciam uma degradação no desempenho do algoritmo *DecisionTable* com o novo conjunto de dados. Esta queda de performance sugere que a eficácia do modelo está

fortemente correlacionada com a distribuição temporal dos dados de treino.

Um desafio evidenciado por este teste prende-se com a variação das rotinas dos passageiros em diferentes alturas do ano. Apoiando a hipótese de que modelos baseados exclusivamente em fatores temporais e de localização são insuficientes para capturar estas variações, procedemos à divisão do conjunto de dados em dois subconjuntos cronológicos: o primeiro, anterior a 25 de maio, e o segundo, posterior a esta data.

Inicialmente, treinámos um modelo utilizando apenas os atributos temporais e de localização com o primeiro conjunto e validámo-lo com o segundo. Como ilustrado na Tabela 4.10, os resultados foram notavelmente incertos, com uma taxa de acerto de apenas 33,72% e um erro absoluto médio de 1,15. Este desempenho reflete a fragilidade de modelos fortemente dependentes de rotinas, algo que muda ao longo do ano.

Tabela 4.10: Matriz de confusão e métricas de erro. Modelo baseado em fatores temporais e localização (antes de 25 de maio). Algoritmo RandomForest.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	244	103	150	81	1	Número total de leituras	2135
2	149	75	82	61	34	Número de previsões erradas	1415
3	133	54	177	182	10	Percentagem de erro	66,28%
4	74	24	64	187	26	Percentagem de acerto	33,72%
5	52	33	60	42	37	Erro médio quadrático	2,482436
						Erro absoluto médio	1,150351
						Erro médio com sinal	-0,04122
						Média do F1-Score	0,327

Para validar a capacidade corretiva dos dados de telemetria, realizámos um segundo teste utilizando o mesmo conjunto de treino, mas incluindo agora os atributos de rede sem fios. Os resultados, apresentados na Tabela 4.11, mostram uma melhoria na eficácia do modelo. Embora a acurácia global se tenha mantido moderada (32,55%), o Erro absoluto médio reduziu-se para 1,07. Este parâmetro é, para o nosso caso de uso, o mais relevante, tratando-se de uma escala de ocupação subjetiva, um erro médio perto de 1 significa que, na prática, o modelo prevê um valor adjacente ao correto, o que é aceitável para o utilizador final.

Com o objetivo de elevar ainda mais o desempenho, realizámos um terceiro teste, enriquecendo o conjunto de dados com novos atributos derivados, as estações de origem e destino, a linha e o sentido dela. Os resultados, sintetizados na Tabela 4.12, demonstram uma evolução clara, com a percentagem de acerto a ascender a 38,76% e o Erro absoluto médio a descer para 0,82, uma redução de 28,7% no Erro absoluto médio face ao primeiro cenário.

Após vários testes, definiu-se que a aplicação final integrará o algoritmo RandomForest devido

Tabela 4.11: Matriz de confusão e métricas de erro. Modelo com fatores temporais e telemetria de rede (após 25 de maio). Algoritmo RandomForest.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	199	133	201	46	0	Número total de leituras	2135
2	62	114	110	82	33	Número de previsões erradas	1440
3	40	55	168	278	15	Percentagem de erro	67,45%
4	49	12	64	211	39	Percentagem de acerto	32,55%
5	28	26	105	62	3	Erro médio quadrático	2,087588
						Erro absoluto médio	1,071194
						Erro médio com sinal	0,233724
						Média do F1-Score	0,316

Tabela 4.12: Matriz de confusão e métricas de erro. Modelo enriquecido com atributos adicionais de viagem. Algoritmo RandomForest.

Previsto	Lido					Resultados e Erros	
	1	2	3	4	5		
1	106	76	55	8	0	Número total de leituras	1473
2	66	70	26	52	1	Número de previsões erradas	902
3	52	44	120	169	8	Percentagem de erro	61,24%
4	8	48	92	246	34	Percentagem de acerto	38,76%
5	2	3	47	111	29	Erro médio quadrático	1,274949
						Erro absoluto médio	0,821453
						Erro médio com sinal	-0,05838
						Média do F1-Score	0,375

à sua superioridade métrica demonstrada em todos os cenários. Especificamente, o modelo utilizará a totalidade dos atributos para previsões completas (quando a telemetria de rede está disponível) e um modelo restringido, contendo apenas os atributos de localização e tempo, para previsões limitadas em cenários de viagem futura ou restrição de dados.

Em suma, embora os resultados demonstrem a eficácia da telemetria na correção da incerteza temporal, a duração limitada do projeto e a volatilidade inerente às rotinas urbanas influenciam a precisão dos modelos. Consideramos que, com uma base de dados que abrangesse um ciclo anual completo, seria possível mitigar estas variações e obter métricas de desempenho consideravelmente superiores.

Capítulo 5

Aplicação Final

5.1 Aplicação final - Descrição

Foi definido como requisito opcional para o projeto a realização de uma aplicação que utilizasse o modelo descrito no Capítulo 4 baseada no conceito de *Crowdsourcing*. Neste âmbito, uma aplicação de *Crowdsourcing* seria definida como uma dependência na coleta de dados anonimizados pelos utilizadores.

O objetivo desta aplicação é fornecer dados em tempo real sobre a ocupação de transportes públicos (especificamente o sistema de comboios e o Metropolitano) na área metropolitana de Lisboa. A funcionalidade principal é prover um mapa interativo dos sistemas de transportes escolhidos com a informação da ocupação atual prevista. Esta informação terá dois níveis de precisão:

- **Previsão limitada:** Durante a fase do treino do modelo verificamos uma precisão de 87,18% do modelo (vide Tabela 4.7) somente com a localização e a hora da viagem. Logo, caso nenhum utilizador consiga fornecer dados de rede para o modelo naquele momento, este modelo específico será utilizado para prover uma estimação.
- **Previsão completa:** Caso um dos utilizadores da aplicação desenvolvida esteja dentro de um transporte, a informação coletada será enviada para o modelo com a maior percentagem de acurácia treinado (vide Tabela 4.5) e será disponibilizado para os outros utilizadores da aplicação.

Para a realização do segundo método de previsão, desenvolveu-se uma prova de conceito para detetar automaticamente se o utilizador está dentro do transporte e realizar a coleta. Foi entendido como uma necessidade prática de forma a não depender que seja feito o início manual das leituras.

5.2 Prova de conceito - Detecção se o utilizador está dentro do transporte

5.2.1 Modelação da Máquina de Estados

Antes de iniciar a fase de desenvolvimento da prova de conceito, foi fundamental conceber um modelo lógico para servir de base para a aplicação. Este modelo foi idealizado sob a forma de uma máquina de estados, sustentada pela correlação entre eventos de localização espacial (*Geofencing*) [41] e reconhecimento de atividade física detectados por duas APIs [42] que serão mencionadas posteriormente. Para a sua implementação, foi necessário mapear geograficamente as estações de metro e comboio da cidade de Lisboa, assim como as suas respetivas entradas, de forma a acionar os eventos de deteção.

A máquina de estados teórica concebida transita entre as seguintes quatro fases principais:

- **Estado Exterior:** Corresponde ao estado inicial ou de repouso. Ocorre quando a aplicação se encontra em segundo plano e o utilizador está numa zona não abrangida pelas estações mapeadas. Neste estado não há qualquer recolha de dados activa.
- **Estado na Estação:** Assumido quando o sistema de *geofencing* emite um evento de entrada (`GEOFENCE_TRANSITION_ENTER`) e o utilizador permanece dentro dessa região (tipicamente associado às atividades de locomoção `WALKING` ou `STILL` por um determinado período). A partir deste momento, o modelo ativa uma verificação de presença temporária e fica a aguardar ativamente que o reconhecimento de atividade transite para um estado que indique deslocação veicular.
- **Estado em Trânsito:** Transicionado no instante em que o mecanismo de reconhecimento deteta a transição de um estado estático ou de caminhada para o estado de deslocação em veículo (`IN_VEHICLE`). Nesta fase, o sistema adquire a confiança de que o utilizador iniciou efetivamente a viagem no transporte público. Cruzando esta informação com a zona de entrada validada no estado anterior, é possível inferir a linha utilizada e confirmar que o utilizador se encontra a transitar no interior do transporte.
- **Estado de Chegada ao Destino:** Validado caso, no decorrer do estado de trânsito, seja emitido um novo evento de entrada numa estação distinta e a atividade do utilizador regresse, de forma sustentada, para os padrões de caminhada ou repouso (`WALKING` ou `STILL`). Este padrão assinala que a pessoa abandonou a carruagem, determinando o encerramento do trajeto e a transição de volta para o estado inicial.

5.2.2 Mapeamento das Estações

Para viabilizar o funcionamento prático da prova de conceito, foi necessário realizar o levantamento geográfico detalhado da infraestrutura de transportes. O mapeamento auto-

matizado das estações foi executado através do envio de um pedido HTTP [43] do tipo POST direcionado à API pública Overpass [44], utilizando especificamente o *endpoint* público `https://overpass-api.de/api/interpreter`.

O corpo do pedido enviado foi estruturado na linguagem de consulta própria do serviço, delimitando a pesquisa geográfica à Área Metropolitana de Lisboa e filtrando os nós, caminhos e relações associados à malha ferroviária urbana, conforme ilustrado no bloco de código seguinte:

```
[out:json];
area["name"="Área Metropolitana de Lisboa"]->.searchArea;
(
  nwr["railway"~"^(station|halt)$"]
    ["station"!="subway"]
    ["station"!="funicular"]
    ["funicular"!="yes"]
    ["light_rail"!="yes"]
    ["tram"!="yes"]
    (area.searchArea);
);
out center;
```

A resposta obtida em formato JSON (*JavaScript Object Notation*) forneceu as coordenadas centrais precisas de cada paragem e estação do perímetro delineado. Estes dados foram posteriormente limpos e compilados para construir a base estática de coordenadas que a aplicação consome para instanciar os objetos de monitorização geográfica.

5.2.3 Implementação da Aplicação

Com o intuito de testar o funcionamento prático e validar a viabilidade desta máquina de estados em tempo real, foi desenvolvida a aplicação móvel de prova de conceito designada por `OTP_IsInTransport_PoC`.

À semelhança da aplicação desenvolvida na fase de coleta de dados, esta prova de conceito foi construída nativamente para o ambiente de desenvolvimento *Android*, aproveitando a linguagem *Kotlin* e estruturando a interface de utilizador através de *Jetpack Compose* [45]. O código-fonte desta aplicação encontra-se hospedado e versionado com os restantes componentes do projeto.

5.2.4 Arquitetura e Módulos da Aplicação

A arquitetura da aplicação serve o propósito de implementar a máquina de estados idealizada. Para tal, foram utilizadas nativamente as APIs da plataforma da própria *Google* (*Activity Recognition* e *Geofencing*) com o objetivo de prover o algoritmo com a capacidade de inferir o contexto físico e geográfico do utilizador.

O núcleo da deteção recorre fundamentalmente a dois mecanismos em paralelo, diretamente mapeados com as exigências do modelo lógico:

- **Reconhecimento de Atividade Física:** Através de serviços integrados de reconhecimento de atividade, a aplicação monitoriza de forma contínua as transições do estado por meio da API que verifica qual o método de locomoção do utilizador (por exemplo, a transição de estar parado, para estar a caminhar, ou para se encontrar num veículo). Esta gestão é efetuada através de rotinas dedicadas que processam os eventos em segundo plano para validar transições cruciais como `WALKING`, `STILL` ou `IN_VEHICLE`.
- **Geofencing de Estações:** Considerando que a simples deteção de movimento em veículo não é evidência suficiente para garantir que o utilizador se encontra na rede de transportes públicos (evitando falsos positivos para um carro, por exemplo), foi implementado o sistema de cercas virtuais (*Geofencing*). O sistema define zonas de limite geográfico em torno de apeadeiros e estações em Lisboa, recorrendo a uma lista de dados geográficos predefinidos de coordenadas, como citado anteriormente. Sempre que o utilizador atravessa os limites destas cercas, o mecanismo de escuta emite notificações de entrada ou saída para atualizar o estado na máquina de estados.

5.2.5 Fluxo de Detecção

O estado unificado do transporte na aplicação que concretiza logicamente os conceitos da máquina de estados, é mantido e processado por uma componente central responsável pela gestão da máquina de estados. A premissa do código reflete o mesmo padrão do modelo teórico: a pessoa tem primeiramente de aceder a uma estação registada (para passar para o estado da estação) e, posteriormente, sofrer uma transição de estado para uma atividade que evidencie a deslocação.

O resultado desta máquina de estados dita as atualizações reativas na interface gráfica principal da aplicação. O ecrã permite ao utilizador constatar se as alterações do fluxo e transições de estado detetadas pelo dispositivo correspondem à realidade experienciada da viagem. A interface da aplicação e o respetivo estado de inferência estão evidenciados na Figura 5.1.

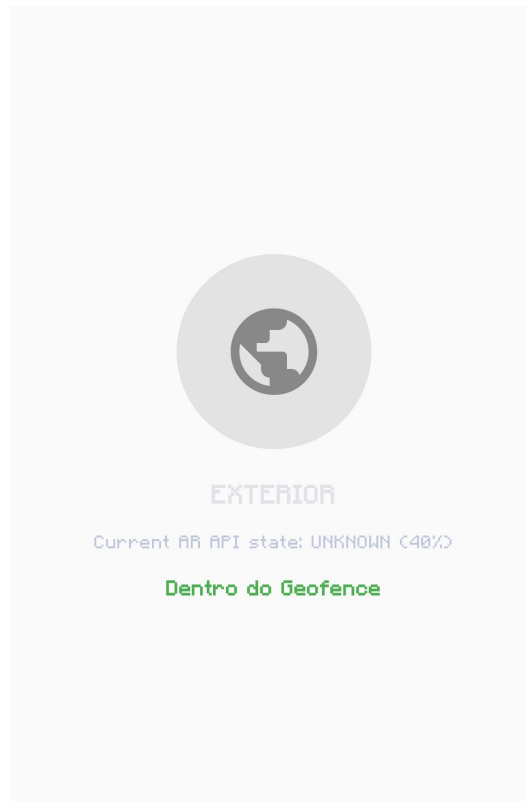


Figura 5.1: Interface gráfica da aplicação `OTP_IsInTransport_PoC` a inferir o estado atual do algoritmo de deteção e transições da máquina de estados.

5.3 Casos de uso

A aplicação final foi concebida para responder a diferentes necessidades dos passageiros no contexto da mobilidade urbana, tirando partido da arquitetura cliente-servidor e das capacidades preditivas do modelo desenvolvido. O primeiro caso de uso principal consiste na consulta da lotação em tempo real. Através de uma interface de mapa interativo integrada na aplicação móvel, o utilizador tem uma visão global da rede de transportes da Área Metropolitana de Lisboa. Ao interagir com o marcador representativo de uma determinada estação de Metropolitano ou Comboio, a aplicação interroga o servidor e exhibe, de imediato, a previsão de ocupação gerada mais recente a partir de outro utilizador para essa localização.

Para além da consulta no momento, o sistema disponibiliza um segundo caso de uso focado no planeamento antecipado de viagens. Neste cenário, um utilizador que pretenda organizar a sua deslocação futura acede a um ecrã dedicado na aplicação. Aqui, seleciona-se o meio de transporte pretendido, a estação de embarque, a data e a hora da viagem. Com base exclusivamente nestes parâmetros, a aplicação submete o pedido ao *backend*, que recorre ao modelo restrito (treinado apenas com informações de horário e de localização) para devolver uma estimativa da ocupação esperada. Esta funcionalidade fornece aos passageiros uma ferramenta de planeamento capaz de os auxiliar a reajustar horários de forma a evitar os períodos de

maior afluência.

O terceiro e mais estrutural caso de uso diz respeito à contribuição passiva de dados (*Crowdsourcing*) que alimenta e sustenta o ecossistema de previsões em tempo real. Ao contrário das abordagens tradicionais, este processo decorre sem necessitar de intervenção manual do utilizador. A aplicação móvel recorre à máquina de estados descrita na Secção 5.2.1 (no contexto da aplicação desenvolvida como prova de conceito) para inferir autonomamente o momento em que o passageiro inicia e decorre numa viagem. Ao confirmar o estado de trânsito, o dispositivo recolhe métricas de rede sem fio em segundo plano e envia os dados de telemetria para o servidor. O *backend* processa imediatamente esta evidência através do modelo de aprendizagem de máquina instanciado, gerando uma nova previsão de ocupação. Este dado é armazenado na base de dados e disponibilizado no mapa interativo para todos os utilizadores da aplicação.

5.4 Servidor Web em *Spring Boot*

De forma a operacionalizar o modelo de aprendizagem de máquina desenvolvido e permitir que as inferências de ocupação sejam consumidas em larga escala, concebeu-se e implementou-se um servidor web de *backend* utilizando a *framework* Spring Boot [46]. Este componente atua como o núcleo da arquitetura do ecossistema de *crowdsourcing*, sendo responsável por expor uma API que faz a ponte entre a recolha de dados dos sensores e os utilizadores finais do sistema.

A arquitetura do servidor foi desenhada seguindo o padrão de desenvolvimento em camadas (*Controller*, *Service* e *Repository*), garantindo o desacoplamento do código e facilitando a manutenção futura do sistema. O fluxo de funcionamento da infraestrutura divide-se em duas vertentes operacionais principais:

- *Ingestão de Métricas Sem Fio*: O servidor disponibiliza *endpoints* HTTP do tipo **POST** concebidos para receber dados em formato JSON submetidos automaticamente pelas instâncias da aplicação móvel em trânsito. Estas mensagens contêm as leituras brutas dos sensores (contagem de dispositivos Bluetooth, intensidades de sinal Wi-Fi, métricas de rede móvel e geolocalização). Ao dar entrada no servidor, os dados são validados e mapeados para Objetos de Transferência de Dados (DTOs - *Data Transfer Objects*) [47].
- *Exposição de Inferências de Ocupação*: Através de pedidos HTTP do tipo **GET**, as aplicações clientes (móveis ou web) podem interrogar o servidor sobre o estado de lotação de uma determinada estação de Lisboa. O Spring Boot processa o pedido, submete as coordenadas ou o horizonte temporal atual ao classificador *Random Forest* integrado, e devolve uma resposta estruturada com o nível de ocupação estimado numa escala

preditiva de 1 a 5.

As responsabilidades são divididas em camadas: a camada de controlo (*Controller*) é estritamente encarregue de gerir comunicação HTTP, validando os dados de entrada e assegurando a correta tradução de formatos através de DTOs. Abaixo desta, a camada de negócio (*Service*) centraliza as regras operacionais da plataforma. É neste nível que ocorre a lógica de negócio do sistema, onde o processamento das mensagens de telemetria invoca, de forma coordenada, os serviços de localização para encontrar a estação mais próxima e o modelo de inferência para gerar uma previsão. Por fim, a camada de persistência (*Repository*) faz o acesso ao armazenamento relacional, encapsulando as operações de leitura e escrita através de mapeamento objeto-relacional realizado através da API de persistência Java (JPA) [48] em sua implementação do ecossistema Spring Boot, Spring Data JPA [49].

Para a manutenção e controlo do volume de dados na base de dados relacional, o *backend* recorre ao mecanismo de escalonamento de tarefas nativo do Spring Boot. Através do módulo de *Scheduling*, implementou-se um serviço dedicado que utiliza a anotação `@Scheduled(fixedRate = 300000)`. Esta diretiva instrui a *framework* a executar uma rotina assíncrona em intervalos exatos de cinco minutos (300 000 milissegundos). Durante cada ciclo de execução, o serviço estabelece um limiar temporal correspondente a trinta minutos no passado e invoca o repositório de dados para emitir uma instrução `DELETE SQL`. Esta operação remove exclusivamente os registos de previsões do tipo `COMPLETE` gerados antes desse limiar.

5.4.1 Descrição dos *endpoints*

O servidor disponibiliza uma interface de programação baseada na arquitetura REST [50] (*Representational State Transfer*), comunicando através do protocolo HTTP e utilizando JSON como formato de serialização de dados. Esta API estabelece a comunicação entre a aplicação móvel, que submete periodicamente as evidências físicas recolhidas, e a interface de visualização do utilizador. Descrevem-se de seguida os *endpoints* disponíveis para acesso via HTTP.

5.4.1.1 Submissão de telemetria

Este *endpoint* é invocado exclusivamente em segundo plano pela aplicação móvel quando a máquina de estados (Secção 5.2.1) deteta que o utilizador se encontra em trânsito. O seu objetivo é receber as leituras de telemetria coletadas pelo utilizador para alimentar o modelo de aprendizagem de máquina instanciado, como mostrado no exemplo 5.1.

- **Método:** POST
- **Caminho:** `/api/v1/telemetry`
- **Parâmetros do *Request*:**

- `timestamp` (*Integer/Int64*): *Timestamp Unix* correspondente ao momento da leitura.
- `latitude` (*Double*): Coordenada de latitude do dispositivo.
- `longitude` (*Double*): Coordenada de longitude do dispositivo.
- `bluetoothCount` (*Integer*): Número total de dispositivos Bluetooth visíveis nas imediações.
- `bluetoothSignals` (*Array de Inteiros*): Lista de 5 valores correspondentes à potência (*RSSI*) dos dispositivos Bluetooth com maior sinal captados.
- `wifiCount` (*Integer*): Número total de redes Wi-Fi visíveis.
- `wifiSignals` (*Array de Inteiros*): Lista de 5 valores com a potência (*RSSI*) das redes Wi-Fi com maior sinal detetadas.
- `rsrp` (*Integer*): *Reference Signal Received Power*, indicando a potência do sinal celular.
- `rssnr` (*Integer*): *Reference Signal Signal-to-Noise Ratio*, indicando a relação sinal-ruído.
- `rsrq` (*Integer*): *Reference Signal Received Quality*, indicando a qualidade do sinal celular.
- `latencyAvg` (*Double*): Latência média de rede (em milissegundos).
- `latencyStdDev` (*Double*): Desvio padrão temporal da latência registada (em milissegundos).
- `packetLoss` (*Double*): Percentagem de perda de pacotes na comunicação (escala em percentagem de 0 a 100).

• **Respostas:**

- 200 OK: Submissão processada com sucesso. Retorna um objeto com o nível de ocupação inferido pelo modelo.
- 400 Bad Request: Erro de validação nos dados submetidos (parâmetros ausentes ou mal formatados).

Listing 5.1: Exemplo do corpo de um *request* de telemetria

```

1 {
2   "timestamp": 1717776000,
3   "latitude": 38.7223,
4   "longitude": -9.1393,
5   "bluetoothCount": 46,
6   "bluetoothSignals": [42, 43, 44, 45, 46],

```

```

7  "wifiCount": 13,
8  "wifiSignals": [-45, -50, -55, -60, -65],
9  "rsrp": -95,
10 "rssi": 15,
11 "rsrq": -8,
12 "latencyAvg": 45.2,
13 "latencyStdDev": 12.1,
14 "packetLoss": 0.5
15 }

```

5.4.1.2 Consulta do mapa de ocupação

Utilizado pelo ecrã principal da aplicação móvel para mapear as estações que possuem uma previsão gerada por algum utilizador nos últimos 30 minutos. Pode ser visualizado no exemplo 5.2.

- **Método:** GET
- **Caminho:** /api/v1/occupancy/map
- **Respostas:**
 - 200 OK: Retorna uma lista contendo as previsões armazenadas do tipo *COMPLETE*.

Listing 5.2: Exemplo de resposta com duas estações

```

1  [
2  {
3    "stationId": "alameda_metro",
4    "name": "Alameda",
5    "latitude": 38.7369,
6    "longitude": -9.1338,
7    "transportType": "METRO",
8    "occupancyLevel": 3,
9    "predictionType": "COMPLETE"
10 },
11 {
12   "stationId": "oriente_train",
13   "name": "Lisboa - Oriente",
14   "latitude": 38.7678,
15   "longitude": -9.0991,
16   "transportType": "TRAIN",
17   "occupancyLevel": 1,
18   "predictionType": "COMPLETE"
19 }
20 ]

```

5.4.1.3 Consulta de ocupação de uma estação

Permite enviar para o modelo mais simples instanciado um pedido sobre qual o nível de lotação de uma estação utilizando apenas localização e horário, permitindo assim verificar viagens

futuras e estações que não tenham sido utilizadas por utilizadores da aplicação recentemente. Exemplo na listagem 5.3.

- **Método:** GET
 - **Caminho:** /api/v1/occupancy/stations/{stationId}
 - **Parâmetros na URL (*Path Variables*):**
 - **stationId** (*String*): O identificador único da estação alvo da consulta.
 - **timestamp** (*Integer*): Horário para que seja realizada a consulta em *Timestamp Unix*
 - **Respostas:**
 - 200 OK: Detalhes da ocupação da estação encontrada.
 - 404 Not Found: A estação especificada não existe no catálogo do sistema.
- }

Listing 5.3: Exemplo de resposta de uma estação

```
1 {  
2   "stationId": "alameda_metro",  
3   "name": "Alameda",  
4   "latitude": 38.7369,  
5   "longitude": -9.1338,  
6   "transportType": "METRO",  
7   "occupancyLevel": 3,  
8   "predictionType": "LIMITED"  
9 }
```

5.4.2 Modelo de Dados

A persistência de dados do sistema foi desenhada em torno de uma base de dados relacional PostgreSQL [51]. O modelo de dados central foca-se em duas entidades primárias: as **Estações** (*stations*) e as **Previsões** (*predictions*), como mostrado na figura 5.2. A tabela de estações atua como o catálogo estático da rede, armazenando a informação geográfica (latitude e longitude), o nome e o identificador do tipo de transporte (Metropolitano ou Comboio) de cada paragem. Por sua vez, a tabela de previsões regista o histórico de inferências de ocupação geradas pelo modelo de aprendizagem de máquina. Esta tabela relaciona-se diretamente com a estação respetiva através de uma chave estrangeira e cataloga cada registo pelo seu tipo de precisão (*Complete* ou *Limited*), armazenando a data de criação. Para otimizar o tempo de resposta nas consultas em tempo real exigidas pela aplicação móvel, foram criados índices compostos na tabela de previsões, ordenando os dados cronologicamente por estação e tipo de transporte.

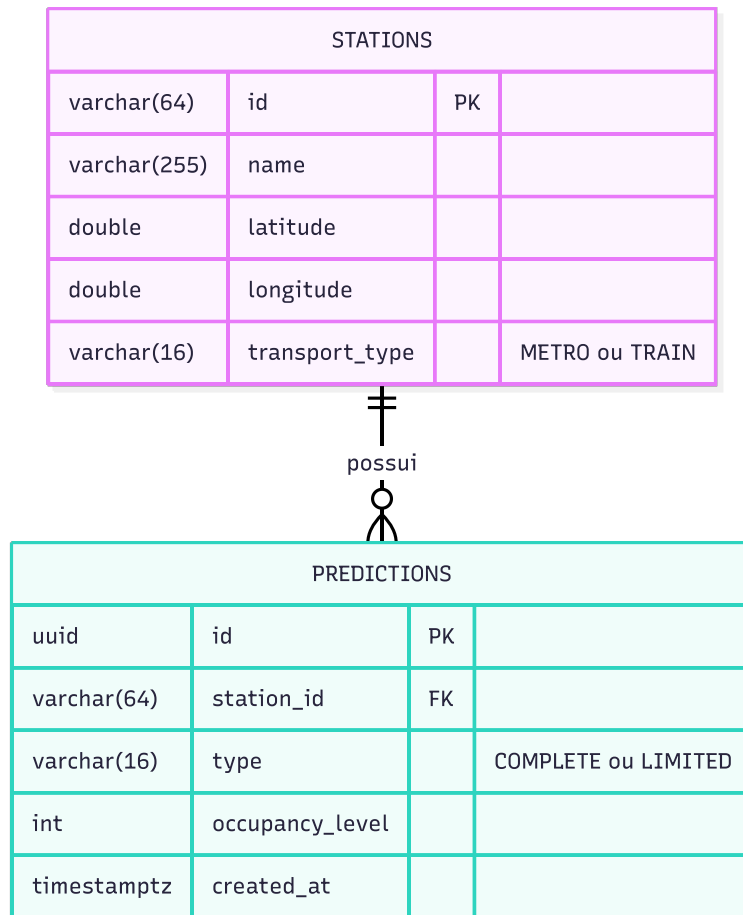


Figura 5.2: Diagrama entidade-relacionamento desenvolvido em mermaid [2, 3]

5.4.3 Infraestrutura e *implantação*

Para a colocação do sistema em ambiente de produção, adotou-se uma estratégia assente em virtualização e contentorização, visando assegurar a disponibilidade do servidor. O *backend* e o sistema de gestão de base de dados foram alojados na plataforma de computação em nuvem *Google Cloud Platform*, recorrendo a uma instância virtual do serviço *Compute Engine* [52] configurada com o perfil de recursos *e2-small* (composta por duas vCPUs partilhadas e 2 GB de memória RAM). Junto com a solução de contentorização foi possível disponibilizar o acesso aos *endpoints* da API do servidor de forma online servida por esta máquina virtual instanciada no GCP.

O processo de instanciação e execução dos serviços foi integralmente estruturado através da plataforma Docker [53] através do *Dockerfile* 5.4, utilizando a ferramenta de orquestração *Docker Compose* [54]. Foram definidos e configurados dois contentores independentes que interagem numa rede virtual privada isolada: o primeiro encarregue de executar a aplicação Spring Boot (empacotada com o ambiente de execução Java e os ficheiros binários dos modelos preditivos) e o segundo dedicado exclusivamente à base de dados PostgreSQL. Esta abordagem

garante que todo o ambiente de execução do ecossistema seja imutável e facilmente replicável através de um único descritor de configuração, eliminando dependências ou conflitos com o sistema operativo anfitrião da máquina virtual.

Listing 5.4: Dockerfile utilizada para contentorização do servidor

```
1 FROM amazoncorretto:23-alpine
2
3
4 WORKDIR /app
5
6 COPY modules/host/build/libs/host.jar app.jar
7
8 EXPOSE 8080
9
10 ENTRYPOINT ["java", "-jar", "app.jar"]
```

5.5 Aplicação Android

5.5.1 Funcionamento do *frontend*

Para operacionalizar os casos de uso descritos, foi desenvolvida uma aplicação *Android* nativa, em *Kotlin*, construída com *Jetpack Compose*. A aplicação estrutura-se em torno de três funcionalidades principais.

A primeira, apresentada no ecrã principal, como mostra a Figura 5.3, consiste num mapa interativo que abrange todas as estações de metro e as principais estações de comboio da Área Metropolitana de Lisboa. O mapa representa a ocupação das estações através de um código de cores intuitivo, como mostra a Figura 5.4: verde para níveis de ocupação 1 ou 2, amarelo para 3 ou 4, e vermelho para o nível 5, permitindo ao utilizador uma avaliação imediata do estado da rede. Estes indicadores baseiam-se em leituras recolhidas por utilizadores que transitaram pela estação recentemente (últimos 30 minutos). Para garantir a atualização, a aplicação recorre à concorrência estruturada do *Kotlin*, mais concretamente às corrotinas [55]. Ao iniciar a aplicação, é lançada uma corrotina no ciclo de vida do *ViewModel* que executa periodicamente pedidos ao servidor, processados pela biblioteca *Ktor* [56], para obter os dados de ocupação mais atuais. Estes dados são expostos à interface através de um *StateFlow*, permitindo que o *Jetpack Compose* reaja de forma reativa e eficiente a cada nova atualização de ocupação.

Esta funcionalidade sustenta-se no conceito central da aplicação: o *Crowdsourcing*. De forma passiva e sem necessidade de intervenção manual, a aplicação deteta autonomamente quando o utilizador se encontra em viagem, recorrendo à lógica de prova de conceito para deteção de trânsito (descrito na Secção 5.2.1). Durante o trajeto, são recolhidas métricas de redes sem fios que, uma vez enviadas para o servidor, são processadas por um modelo de aprendizagem de máquina. O resultado é uma estimativa de ocupação em tempo real, disponível para toda

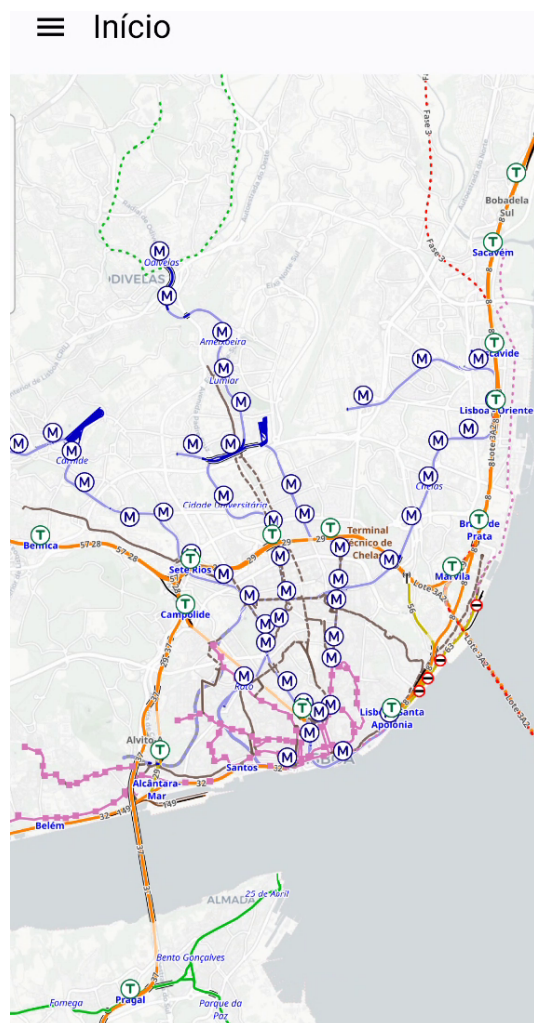


Figura 5.3: Mapa, sem previsões, apresentado no ecrã principal da aplicação final.

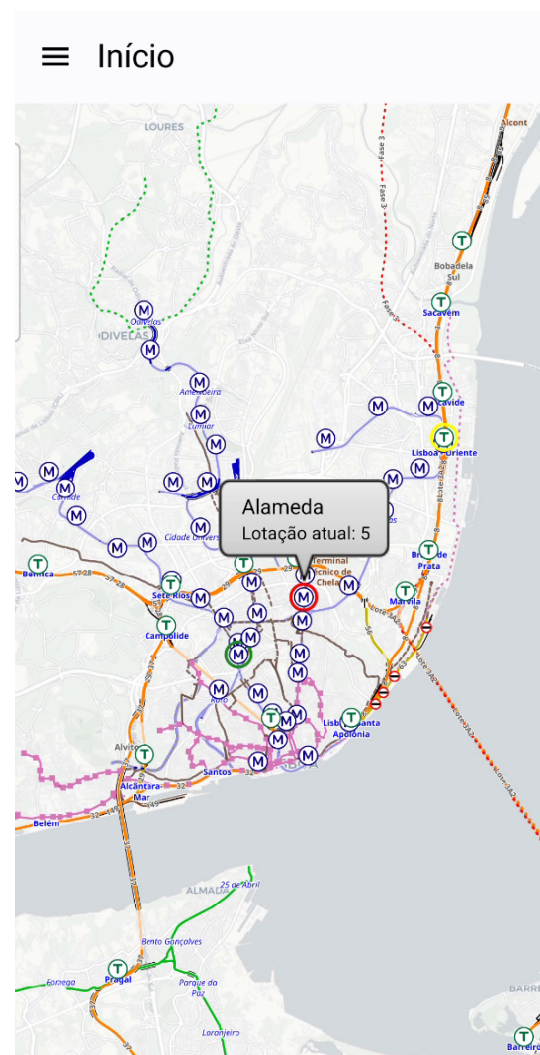


Figura 5.4: Mapa, com previsões, apresentado no ecrã principal da aplicação final.

a comunidade.

Considerando que a ausência de utilizadores ativos em determinadas zonas ou períodos poderia limitar a utilidade do sistema, desenvolvemos uma terceira funcionalidade: um sistema de previsão baseado em padrões horários. Embora este modelo apresente uma margem de incerteza superior ao método baseado em dados de rede, revela-se robusto para a generalidade dos dias. Sempre que é selecionada uma estação que não possui dados recolhidos por utilizadores nos últimos 30 minutos, é efetuado um pedido ao servidor para obter uma previsão baseada exclusivamente em padrões temporais. Adicionalmente, num segundo ecrã, apresentado na Figura 5.5, o utilizador pode planear viagens futuras selecionando o meio de transporte, a estação, o dia e a hora, obtendo uma estimativa de ocupação prevista, apresentada na Figura 5.6.

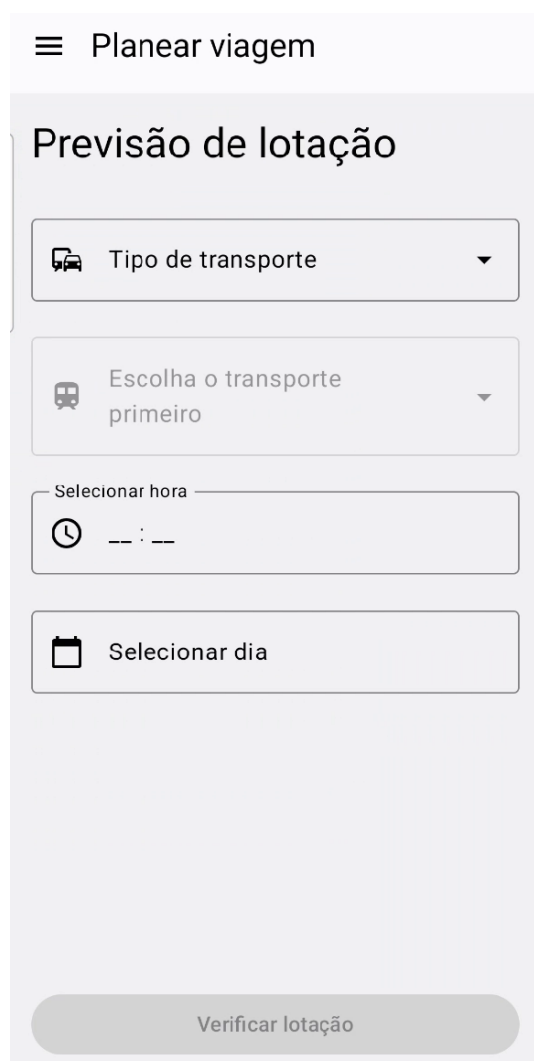


Figura 5.5: Ecrã para planear viagem da aplicação final.

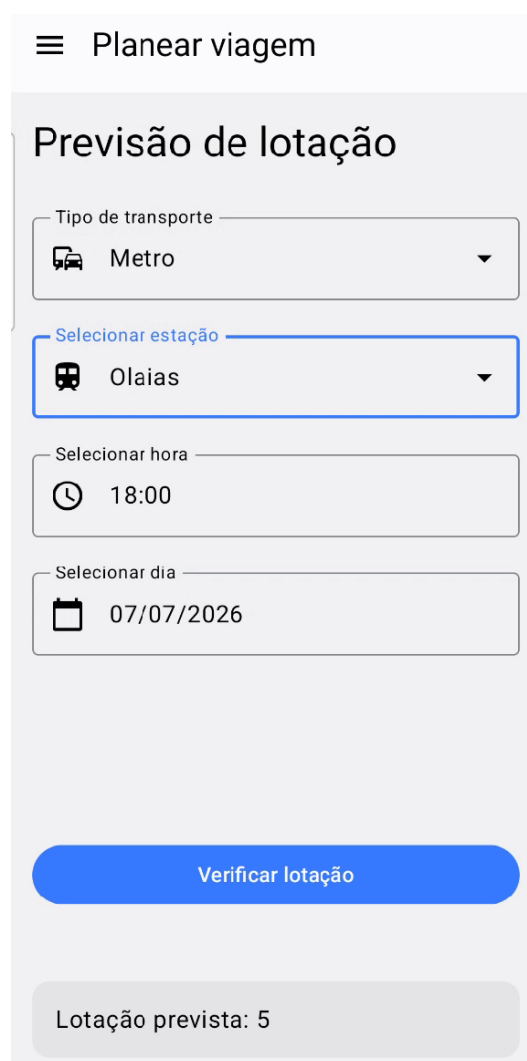


Figura 5.6: Ecrã para planear viagem, com previsão, da aplicação final.

5.5.2 Diagrama de serviços UML do sistema completo

A figura 5.7 mostra através de um diagrama UML, o fluxo de funcionamento do sistema.

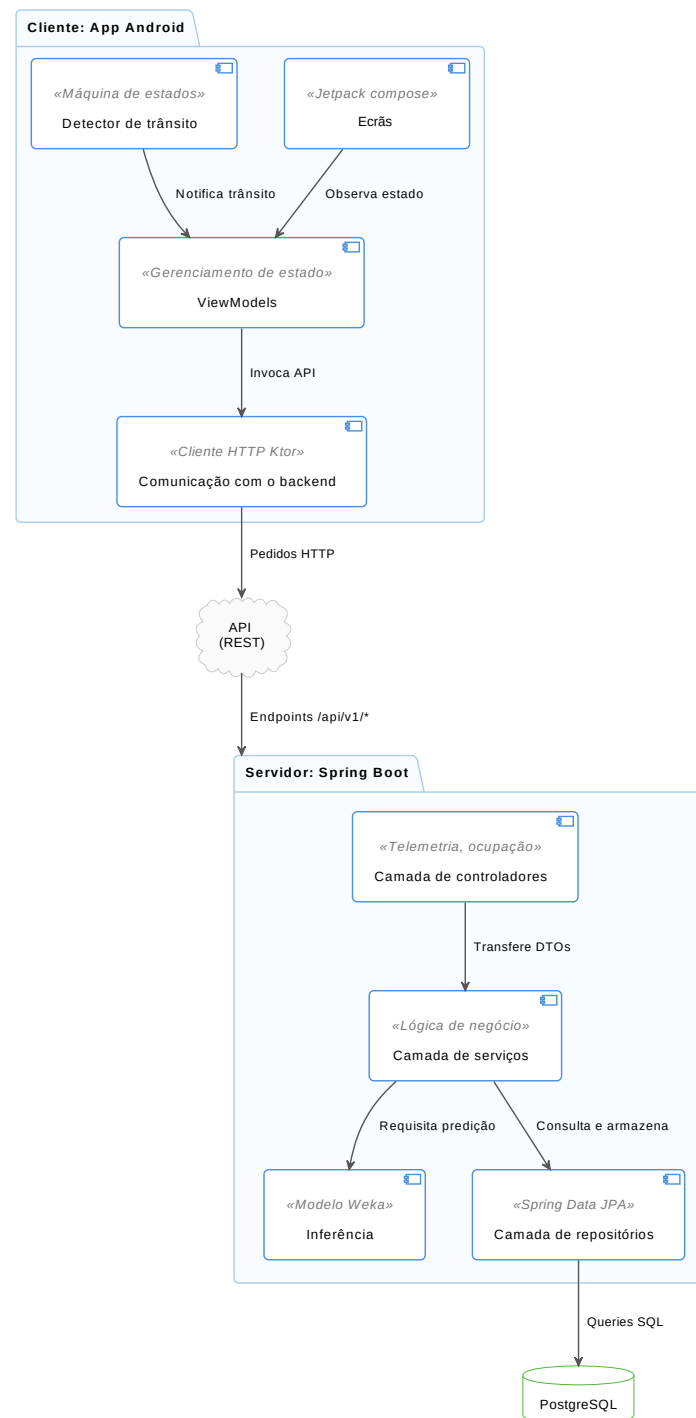


Figura 5.7: Diagrama de arquitetura do sistema de estimação de ocupação de transporte.

5.5.3 Algoritmo de detecção de trânsito

O algoritmo implementado na prova de conceito para detecção de trânsito (descrito na Secção 5.2.1) também foi efetivado na aplicação Android final, atuando como parte essencial do processo de enviar telemetria para o servidor e realizar uma previsão para o mapa. Garante que as transições de estado e o consequente começo da recolha ativa de telemetria ocorram de forma totalmente automatizada e transparente para o utilizador. Com o objetivo de documentar e clarificar a dinâmica de transição deste algoritmo de detecção passiva, a Figura 5.8 ilustra o diagrama de transição de estados [57] completo.

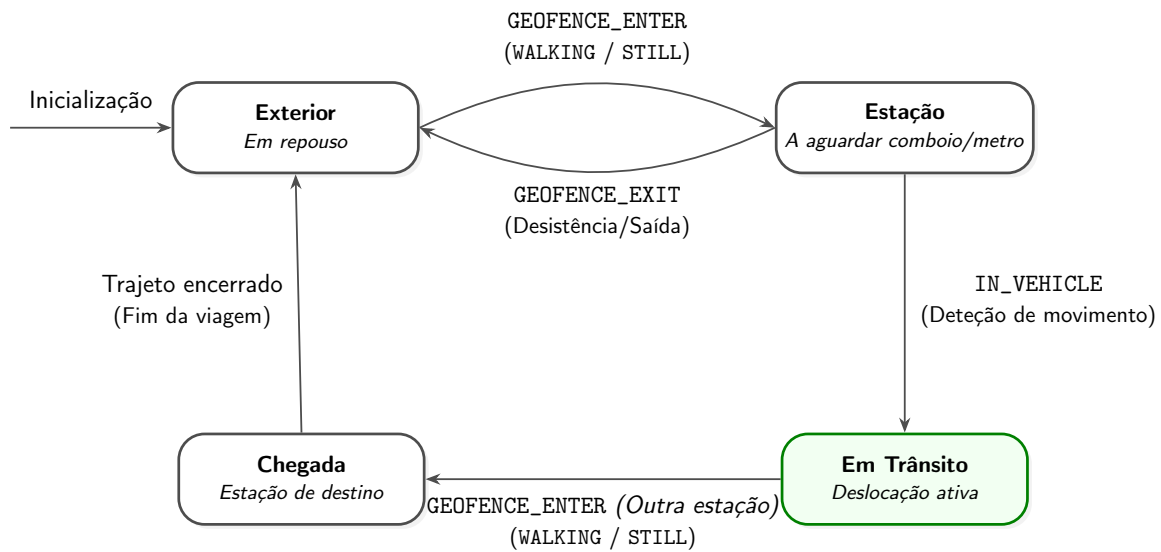


Figura 5.8: Diagrama de transição de estados otimizado para o algoritmo de detecção de trânsito.

5.6 Validação de *software* e testes

Como foi definido na proposta deste projeto como requisito e de forma a garantir a fiabilidade do sistema, a fase de desenvolvimento foi acompanhada pela criação de testes automáticos. Os testes foram divididos entre as componentes de servidor e a aplicação móvel.

5.6.1 Testes do servidor

A arquitetura de testes do *backend* foi desenvolvida com recurso à *framework* JUnit 5 [58], acompanhando o padrão de separação em camadas da aplicação *Spring Boot*. A validação foi segmentada em quatro níveis operacionais distintos:

- **Camada de Domínio (*Domain*):** Focada na validação unitária pura, sem injeção de dependências do ecossistema Spring. Estes testes garantem o comportamento determinístico das entidades primárias, Objetos de Transferência de Dados (DTOs) e das enumerações do sistema (como `TransportType` e `PredictionType`), recorrendo a asser-

ções fundamentais.

- **Camada de Persistência (*Repository*):** Através da anotação `@DataJpaTest`, instanciou-se dinamicamente uma base de dados em memória (H2) para isolar e validar a interação com os dados. Esta suite certifica as operações CRUD, as *queries* derivadas e personalizadas (`@Query`), bem como as restrições de integridade estrutural da base de dados.
- **Camada de Negócio (*Services*):** Recorrendo à biblioteca Mockito [59], isolou-se a lógica de negócio de dependências externas. Foram validados os serviços críticos de processamento de telemetria e mapeamento de ocupação. De particular relevo é o teste de inspeção do modelo algorítmico, que carrega os ficheiros reais do classificador Weka instanciado e assegura que este devolve um classificador.
- **Camada de Apresentação (*Controllers*):** A interface REST foi validada através de testes de integração com a ferramenta `MockMvc`. Com as dependências de negócio simuladas (`@MockitoBean`), testou-se o contrato HTTP dos *endpoints*, assegurando a correta validação estrutural dos corpos dos pedidos (`@Valid`), a coerência das respostas JSON (via *jsonPath*) e a emissão dos devidos códigos de estado HTTP em cenários de sucesso e erro.

5.6.2 Testes da aplicação móvel

No contexto da aplicação Android, a estratégia de testes incidiu sobre a preservação do estado da interface e o processamento assíncrono. Foram implementadas suites de validação para as funções utilitárias e para o modelo de domínio (*Domain Entities* e *Data Models*), garantindo a correta interpretação das respostas do servidor. Destacam-se os testes aplicados aos *ViewModels* e à execução de corrotinas, fundamentais para não bloquear a *thread* principal da interface. O fluxo de recolha de dados foi validado examinando as transições exatas da máquina de estados visual. Adicionalmente, implementou-se uma validação exaustiva do catálogo estático de estações, atestando a presença e ordenação rigorosa das paragens mapeadas na rede metropolitana.

5.6.3 Cobertura

Em suma, a convergência destas abordagens resultou numa bateria abrangente de centenas de testes automatizados (combinando *frontend* e *backend*), executáveis autonomamente através de rotinas *Gradle* [60] (`gradlew test`). Esta densidade de testes reflete um elevado índice de cobertura de código (*Code Coverage*) sobre as rotas críticas do sistema em particular na submissão de telemetrias e nas respostas do modelo de inferência.

Capítulo 6

Conclusão

Em conclusão, o desenvolvimento deste projeto demonstrou com sucesso a viabilidade da utilização da análise passiva de redes sem fio em conjunto com técnicas de aprendizagem de máquina para determinar a ocupação em transportes públicos com certa precisão. A metodologia implementada viabilizou a recolha de um conjunto de dados substancial e consideravelmente representativo, o qual permitiu treinar e estabilizar um modelo assente no algoritmo Random Forest capaz de classificar a lotação subjetiva com acurácia. Para além da componente preditiva, a aplicação móvel que implementa o modelo baseada em *crowdsourcing* validou, na prática, a automatização da deteção de viagens e da partilha destas métricas em tempo real. Desta forma, a abordagem e o ecossistema propostos cumprem integralmente os objetivos delineados no âmbito do projeto, afirmando-se como uma alternativa eficaz, escalável e de custo reduzido face aos sistemas de monitorização tradicionais.

6.1 Requisitos Cumpridos

Foram cumpridos todos os requisitos obrigatórios do projeto, o desenvolvimento da aplicação de coleta de dados; o desenvolvimento e treinamento de um modelo de aprendizagem de máquina; a validação e avaliação deste modelo; tal como os testes unitários correspondentes. Além destes, foram cumpridos vários requisitos opcionais, tal como o desenvolvimento e treinamento utilizando vários algoritmos e dados diferentes; e a aplicação de *Crowdsourcing*, que embora não tenha ficado totalmente concluída, está num estado funcional, e que poderia ser distribuída e utilizada.

6.2 Trabalhos Futuros

Embora tenham sido alcançados os requisitos obrigatórios e opcionais do projeto, existem diversas melhorias que poderiam ter sido feitas. Ao nível da precisão do algoritmo de deteção de trânsito, poderia ter sido encontrado alguma solução mais complexa e fiável; A dependência dos dados da subjetividade de cada voluntário; Para atingir um modelo mais preciso,

idealmente seria necessário uma coleta realizada ao longo de um ano todo para tomar em conta a sazonalidade da utilização dos transportes públicos.

Além disso, a aplicação final que integra o modelo poderia ter tido uma implementação mais escalável. Num cenário com centenas de milhares de utilizadores concorrentes o nosso sistema enfrentaria estrangulamento por conta da única instância do modelo e da base de dados. Uma solução superior seria a implementação de micro-serviços automaticamente escaláveis com recurso a tecnologia muito utilizada no mercado, *Kubernetes*.

Referências

- [1] Google. Gemini. <https://gemini.google.com/>, 2026. [Acedido em: 03-Julho-2026].
- [2] Mermaid Contributors. Mermaid: Diagramming and charting tool. <https://mermaid.js.org/>, 2026. [Acedido em: 03-Julho-2026].
- [3] Peter Pin-Shan Chen. The Entity-Relationship Model: Toward a Unified View of Data. *ACM Transactions on Database Systems (TODS)*, 1(1):9–36, 1976. [Artigo original que introduziu os diagramas Entidade-Relacionamento].
- [4] Autoridade da Mobilidade e dos Transportes (AMT). Observatório AMT: Indicadores da Procura do Transporte Ferroviário de Passageiros (CP e Fertagus). <https://observatorio.amt-autoridade.pt/publicacao/Procura-Anual-Transporte-Procura>, 2025. [Acedido em: 25-Maio-2026].
- [5] Transportes Metropolitanos de Lisboa (TML). Relatório e Contas 2024: Dados sobre Procura, Validações e Oferta de Transporte na AML. Technical report, TML – Autoridade de Transportes da Área Metropolitana de Lisboa, 2025. [Acedido em: 25-Maio-2026].
- [6] Metropolitano de Lisboa, E.P.E. O Metro em Números: Dados Estatísticos de Operação e Procura. <https://www.metrolisboa.pt/institucional/conhecer/metro-em-numeros/>, 2026. [Acedido em: 25-Maio-2026].
- [7] Zheng Li and David A. Hensher. The equity of public transport crowding exposure. *Journal of Transport Geography*, 2013. [Acedido em: 25-Maio-2026].
- [8] Alejandro Tirachini, David A Hensher, and John M Rose. Crowding in public transport systems: Effects on users, operation and implications for the estimation of demand. *Transportation Research Part A: Policy and Practice*, 53:36–52, 2013.
- [9] Edoardo Longo, Alessandro E. C. Redondi, and Matteo Cesana. Accurate occupancy estimation with WiFi and bluetooth/BLE packet capture. *Computer Networks*, 163:106876, 2019.
- [10] Rui Zhou et al. Device-free crowd counting with WiFi channel state information and deep neural networks. *Wireless Networks*, 2020.

- [11] Yongsen Ma, Gang Zhou, and Shuangquan Wang. Wifi sensing with channel state information: A survey. *ACM Computing Surveys (CSUR)*, 52(3):1–36, 2019.
- [12] S. Liu, Y. Zhao, F. Xue, B. Chen, and X. Chen. DeepCount: Crowd Counting with WiFi via Deep Learning. *arXiv preprint arXiv:1903.05316*, 2019.
- [13] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [14] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995.
- [15] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *nature*, 521(7553):436–444, 2015.
- [16] Comboios de Portugal, E.P.E. Projeto de Informação em Tempo Real: Sistema de Monitorização e Operação Ferroviária. <https://cp.pt/info/w/institucional/real-time-info-project>, 2024. [Acedido em: 24-Maio-2026].
- [17] Google Business Profile Help. Popular times, waiting times, and visit duration. <https://support.google.com/business/answer/6263531>, 2026. [Acedido em: 24-Maio-2026].
- [18] Transport for London (TfL). Introduction to NUMBAT: London Underground Passenger Data and Crowding Models. https://crowding.data.tfl.gov.uk/NUMBAT/Intro_to_NUMBAT.pdf, 2023. [Acedido em: 24-Maio-2026].
- [19] MobilityData. General Transit Feed Specification (GTFS) Reference. <https://gtfs.org/documentation/schedule/reference/>, 2026. Acedido a 25 de maio de 2026.
- [20] Metropolitano de Lisboa. API do Metro de Lisboa. <https://api.metrolisboa.pt/store/apis/info?name=EstadoServicoML&version=1.0.1&provider=admin>, 2026. Acedido a 27 de maio de 2026.
- [21] João Pereira. CP Realtime. <https://comboios.live/>, 2026. 2026-05-18.
- [22] Cláudia Conceição and João Almeida. Próximo Metro. <https://proximometro.pt/>, 2026. 2026-05-18.
- [23] Google. Google Firebase Firestore Documentation, 2026. Acedido a 25 de maio de 2026.
- [24] Google. Android Studio, 2026. Acedido a 25 de maio de 2026.
- [25] Google. Wi-Fi Scanning Overview - Wi-Fi Scan Throttling, 2026. Acedido a 25 de maio de 2026.
- [26] Google. Connectivity - Android Developers, 2026. Acedido a 25 de maio de 2026.

- [27] 3GPP. Evolved Universal Terrestrial Radio Access (E-UTRA); Physical layer; Measurements. Technical Specification TS 36.214, 3rd Generation Partnership Project (3GPP), 2026.
- [28] Google. Google Public DNS, 2026. Acedido a 25 de maio de 2026.
- [29] IPutils Project. *ping(8) - Linux manual page*, 2024. Acedido a 25 de maio de 2026.
- [30] Wikipedia. Princípio de Pareto — Wikipedia, the free encyclopedia. <http://pt.wikipedia.org/w/index.php?title=Princ%C3%ADpio%20de%20Pareto&oldid=70751726>, 2026. [Online; accessed 21-May-2026].
- [31] JetBrains. Kotlin Programming Language, 2026. Acedido a 25 de maio de 2026.
- [32] James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley. *The Java language specification*. Addison-Wesley Professional, 2014.
- [33] Oracle. Java Foundation Classes (JFC) - Swing, 2026. Acedido a 25 de maio de 2026.
- [34] University of Waikato. Attribute-Relation File Format (ARFF), 2026. Acedido a 25 de maio de 2026.
- [35] The University of Waikato. Weka Documentation. <https://ml.cms.waikato.ac.nz/weka/>, 2026. [Acedido em: 24-Maio-2026].
- [36] IEEE and The Open Group. The Open Group Base Specifications Issue 7, 2018 edition - IEEE Std 1003.1-2017 (POSIX.1), 2018. Acedido a 25 de maio de 2026.
- [37] GitHub, Inc. GitHub: Where the world builds software, 2026. Acedido a 25 de maio de 2026.
- [38] Python Software Foundation. Python Reference Manual, 2026. Acedido a 25 de maio de 2026.
- [39] Roger W Sinnott. Virtues of the haversine. *Sky and Telescope*, 68(2):159, 1984.
- [40] Rensis Likert. A technique for the measurement of attitudes. *Archives of Psychology*, 22(140):1–55, 1932.
- [41] Google. Geofencing API - Android Developers, 2026. Acedido a 25 de maio de 2026.
- [42] Google. Activity Recognition API - Android Developers, 2026. Acedido a 25 de maio de 2026.
- [43] Roy T. Fielding, Mark Nottingham, and Julian Reschke. Http semantics. RFC 9110, Internet Engineering Task Force (IETF), Junho 2022.

- [44] OpenStreetMap contributors. Overpass API - OpenStreetMap Wiki, 2026. Acedido a 25 de maio de 2026.
- [45] Google. Jetpack Compose - Android Developers, 2026. Acedido a 25 de maio de 2026.
- [46] VMware, Inc. Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>, 2026. [Acedido em: 03-Julho-2026].
- [47] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. Padrão DTO documentado também online em: <https://martinfowler.com/eaaCatalog/dataTransferObject.html>.
- [48] Eclipse Foundation. Jakarta persistence specification, 2026. Acedido em: 05-Julho-2026.
- [49] Spring Data Team. Spring data jpa - reference documentation, 2026. Acedido em: 05-Julho-2026.
- [50] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000. [Tese fundacional que define a arquitetura REST].
- [51] PostgreSQL Global Development Group. *PostgreSQL 15 Documentation*, 2026. [Acedido em: 03-Julho-2026].
- [52] Google Cloud. Compute Engine Documentation. <https://cloud.google.com/compute/docs>, 2026. [Acedido em: 03-Julho-2026].
- [53] Docker Inc. Docker Documentation. <https://docs.docker.com/>, 2026. [Acedido em: 03-Julho-2026].
- [54] Docker Inc. Docker Compose Overview. <https://docs.docker.com/compose/>, 2026. [Acedido em: 03-Julho-2026].
- [55] JetBrains. Coroutines guide | kotlin, 2026. Acedido em: 05-Julho-2026.
- [56] JetBrains. Ktor: Framework for building asynchronous servers and clients, 2026. Acedido em: 05-Julho-2026.
- [57] David Harel. Statecharts: A visual formalism for complex systems. *Science of computer programming*, 8(3):231–274, 1987. [Referência académica clássica para Máquinas de Estados / Statecharts].
- [58] JUnit Team. Junit 5 user guide, 2026. Acedido em: 05-Julho-2026.
- [59] Mockito Contributors. Mockito framework site, 2026. Acedido em: 05-Julho-2026.
- [60] Gradle Inc. Gradle build tool documentation, 2026. Acedido em: 05-Julho-2026.